

Sciences &
Technologies

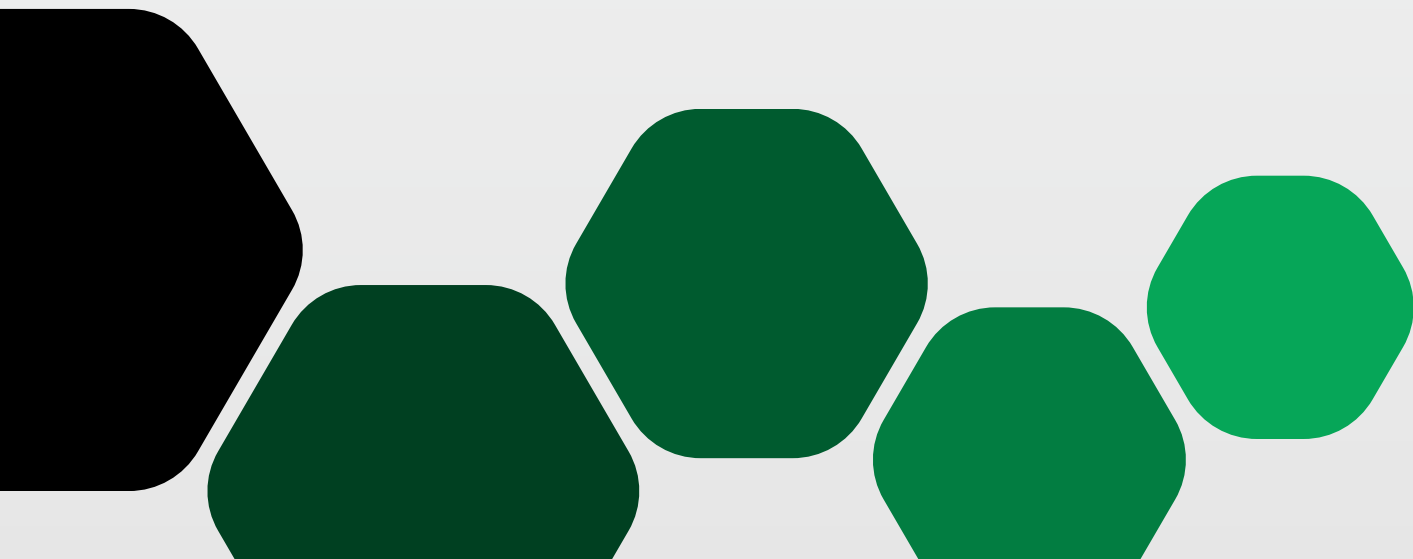


Groupe : Lestandi Théo, Soetens Lukas, Ton That Lavarini Félix,
Theodore--Delage Milan, Guillorit Mathys

AGON

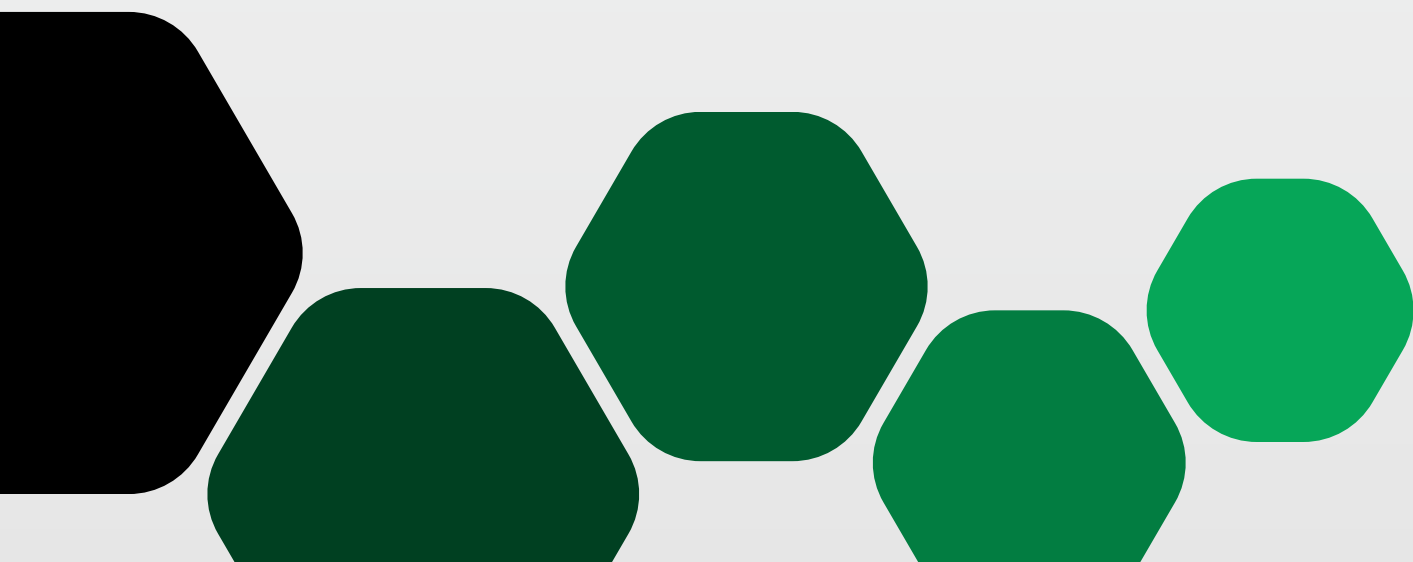
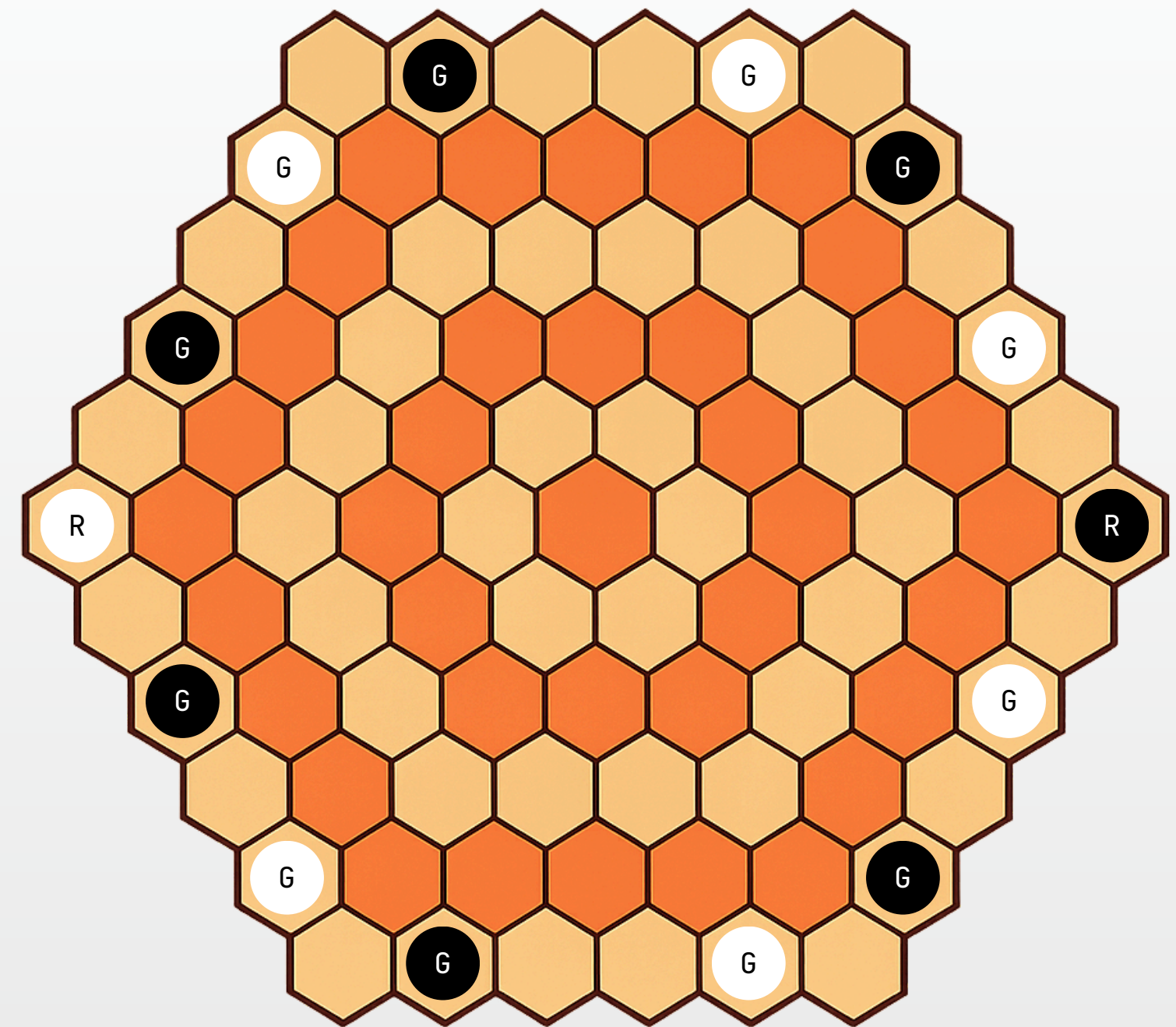
Sommaire

- Règles du jeu
- Présentation du projet : Objectifs & contraintes
- Répartition du travail
- Architecture logicielle
- Arborescence globale
- Zoom sur couches:
 - UI
 - Application
 - AgonCore
 - Technical
- Tests & performances



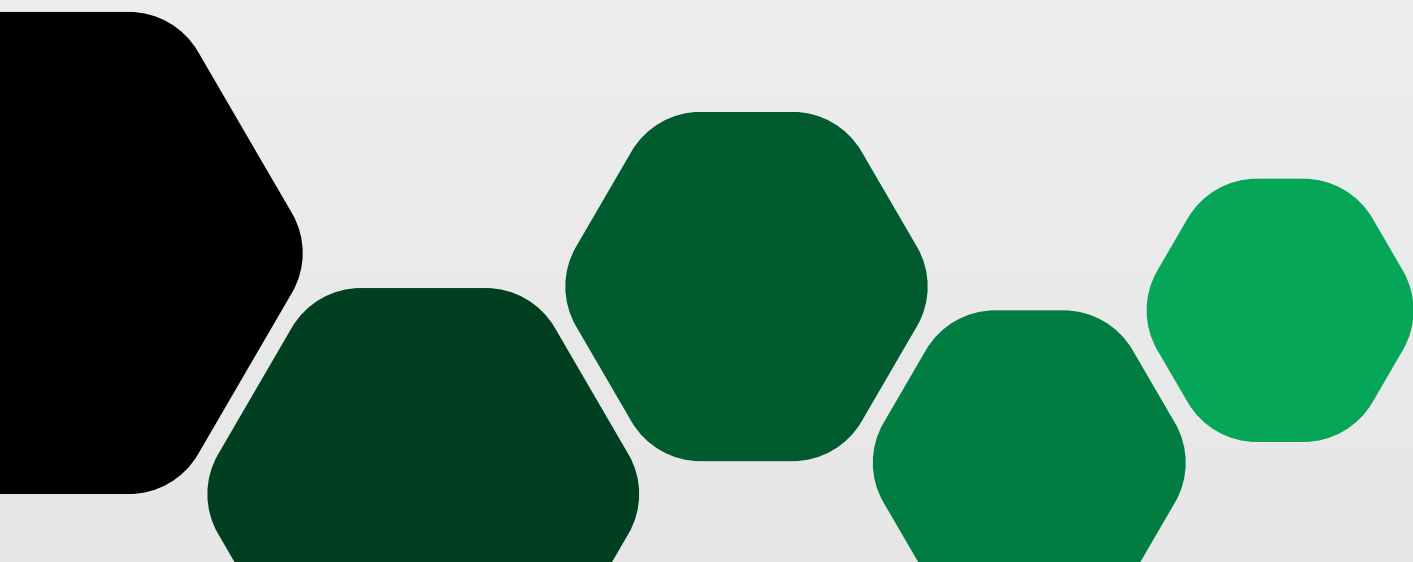
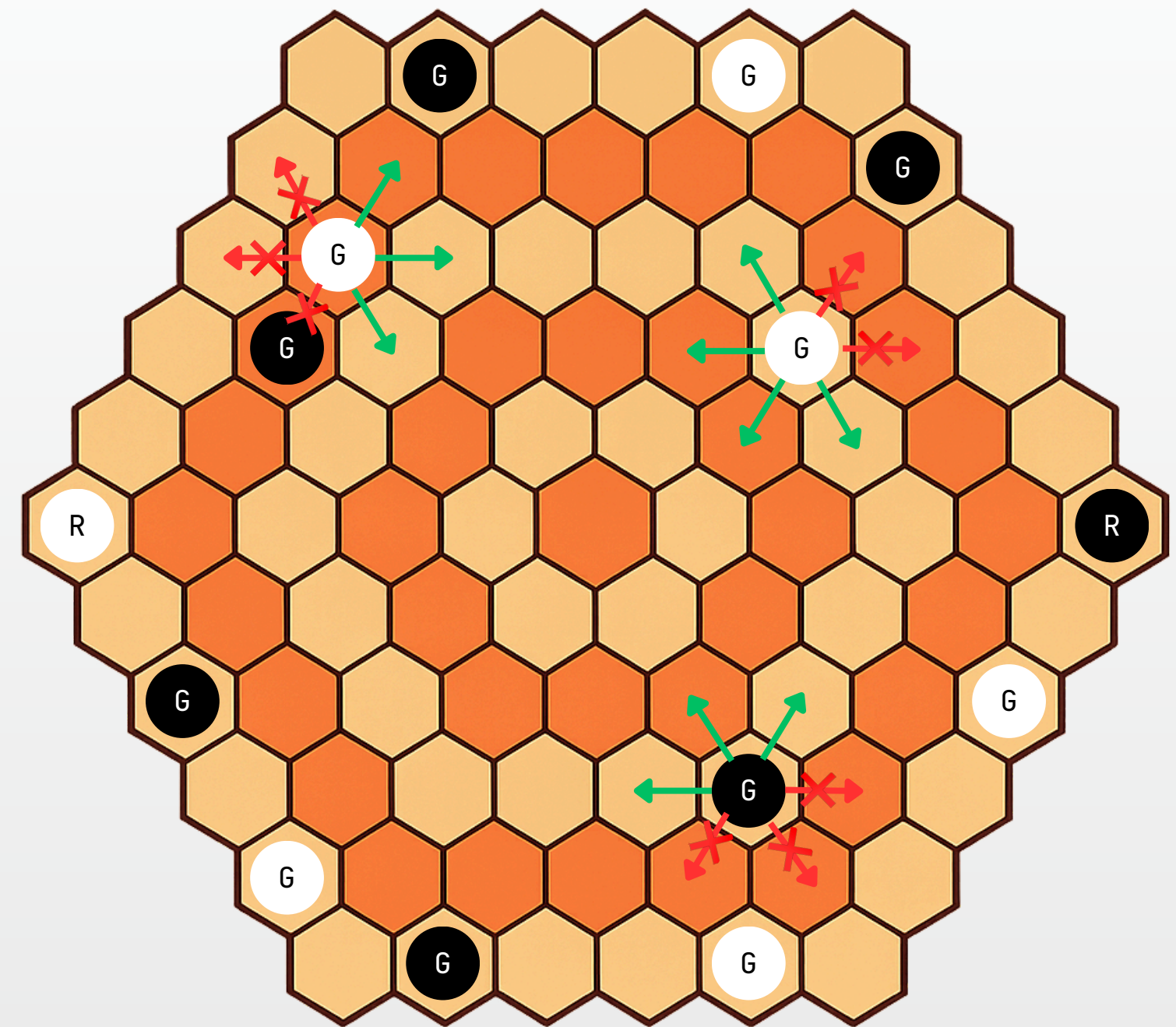
AGON Règles du jeu : bases

- Jeu constitué de 2 équipes (pion blanc et noir)
- Chaque équipe possède :
 - 6 gardes → 
 - 1 reine → 
- Placement initial symétrique

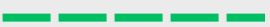
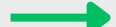
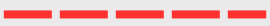


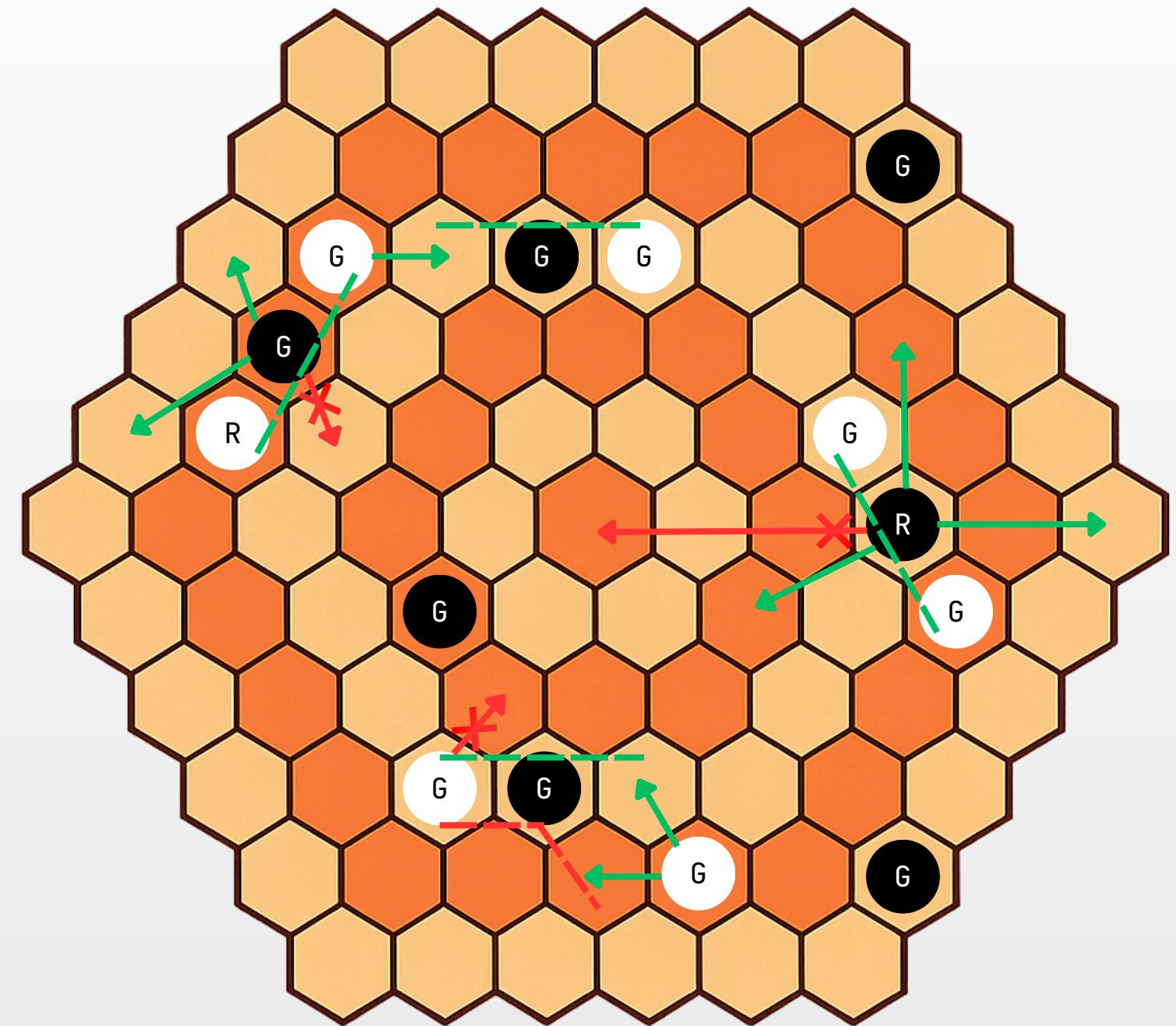
AGON Règles du jeu : déplacements

- Chaque pion peut avancer sur une de ses cases voisines (1 en 1)
- Un pion une fois avancer ne peut revenir en arrière (déplacement à l'avant ou sur les cotés de la position actuelle)
- Interdiction de se déplacer sur une case qui est déjà occupée
- Mouvement :
 - autorisé →
 - non autorisé ↗



AGON Règles du jeu : captures

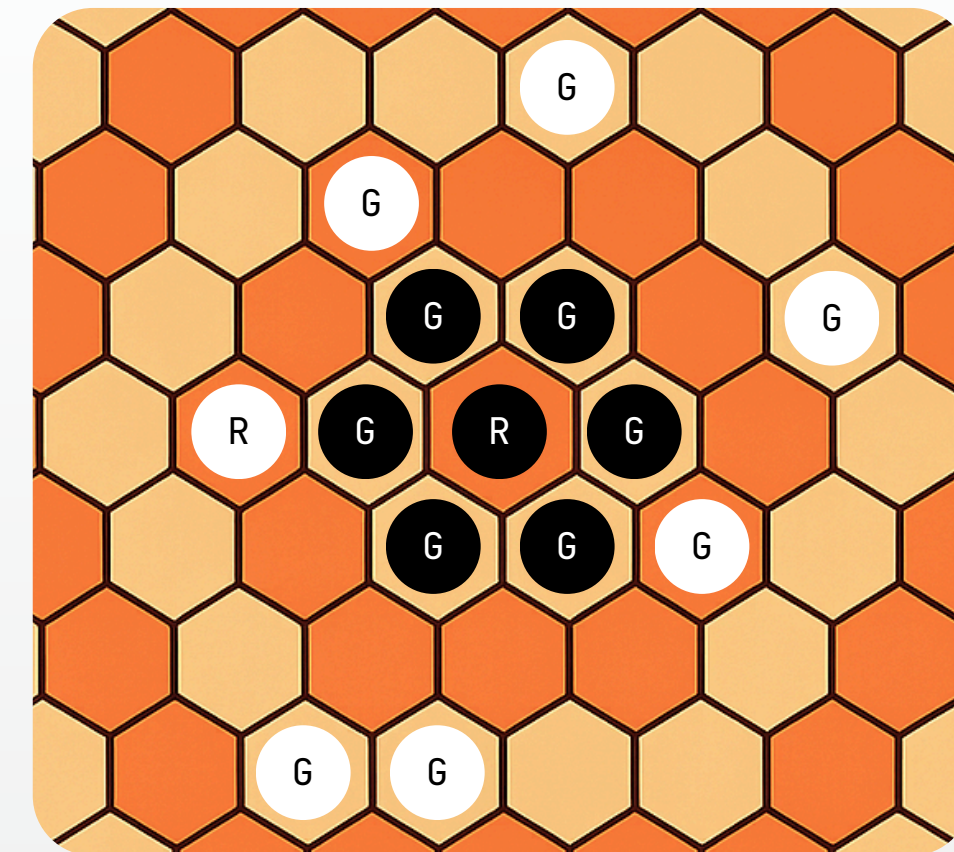
- 2 types de captures:
 - capture d'un garde (l'adversaire replace son garde capturé sur une des cases extérieures qui est non occupé)
 - capture d'une reine (l'adversaire replace sa reine capturée sur une des cases du plateau qui est non occupé à l'exception de la case centrale)
- Un pion est dit capturé si il est encerclé par 2 pions adverse avec un angle de 180°
- Multiples captures autorisées (si on capture un pion adverse, on a la possibilité de rejouer ensuite)
- Interdictions de déplacer son pion vers une position de capture (pour soi ou pour l'adversaire)
- Captures + déplacements :
 - correcte  + 
 - non correcte  + 



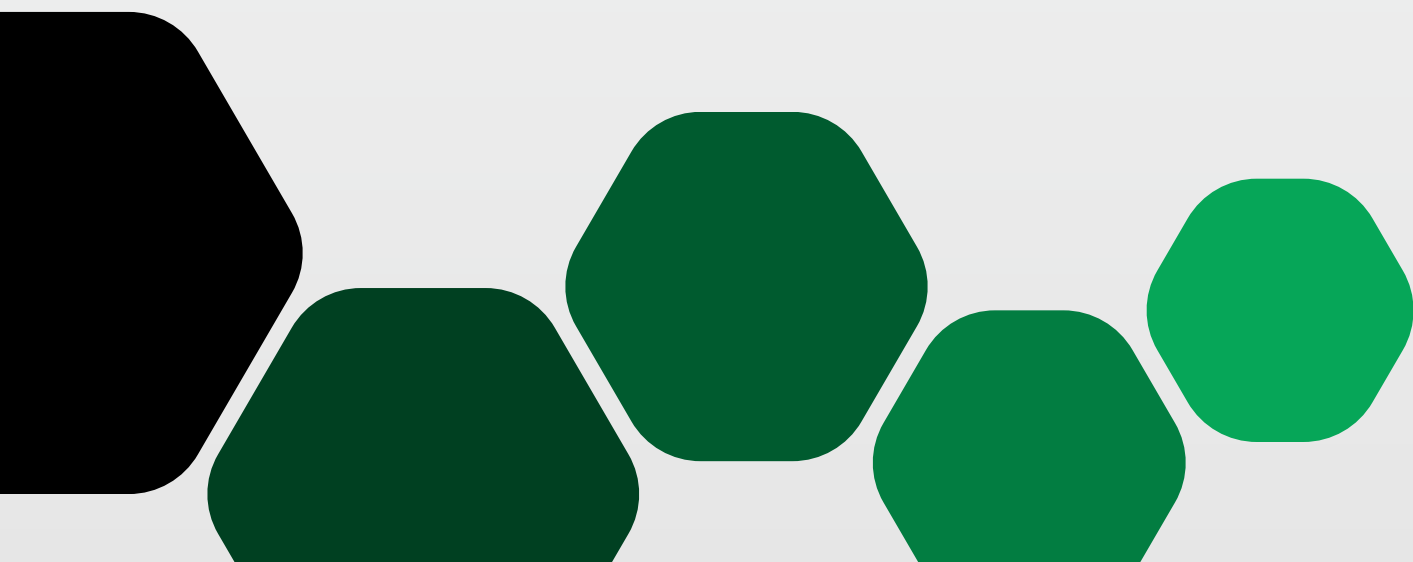
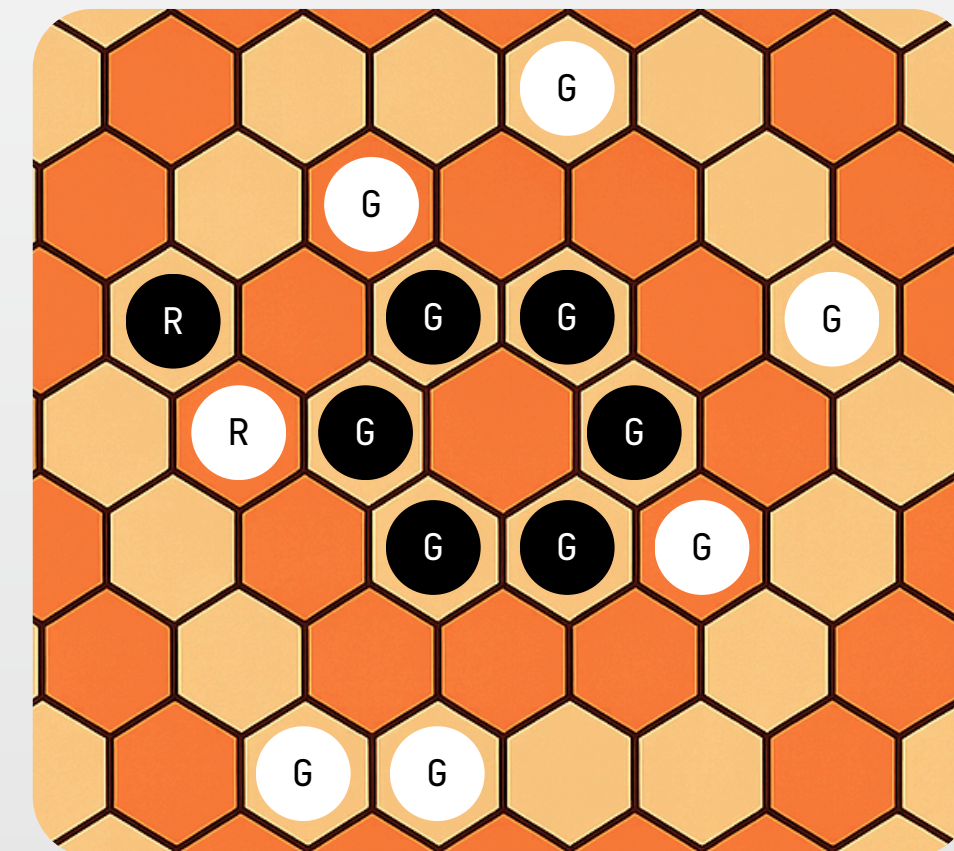
AGON Règles du jeu : fin de partie

- Conditions de victoire : Cas 1
 - Si le joueur arrive à placer sa reine au centre du plateau et que ses 6 gardes l'encerclent
 - L'adversaire n'a plus de coups possibles à jouer
- Conditions de défaite : Cas 2
 - le joueur arrive à former le cercle autour du centre avec ses gardes, mais sa reine n'est pas positionnée au centre du plateau, victoire pour l'adversaire

Cas 1 : Victoire joueur pions noirs



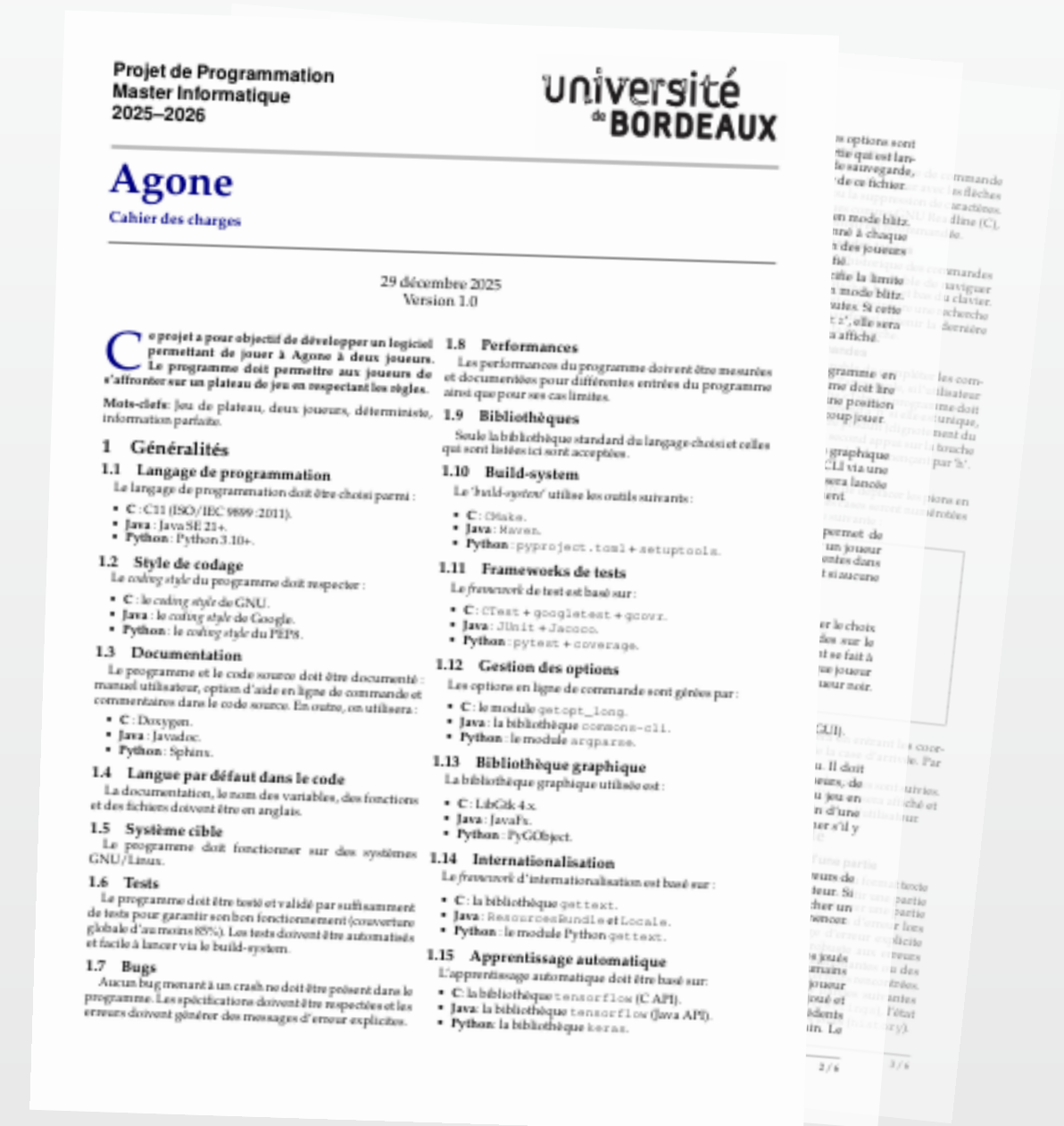
Cas 2 : Défaite joueur pions noirs



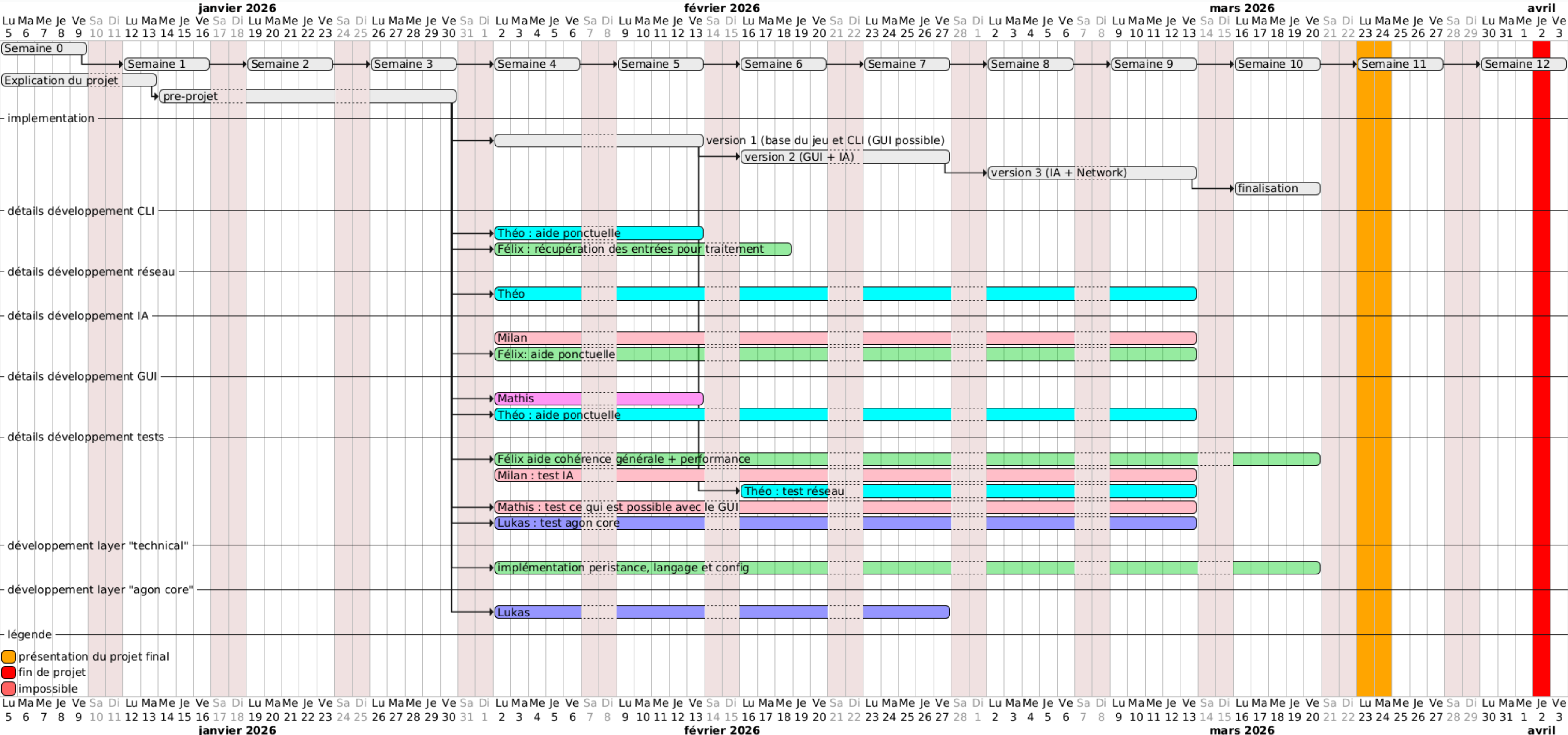
AGON

Présentation projet : Objectifs & contraintes

- Objectifs :
 - Implementation jeu AGON pour 2 joueurs + respect des règles du jeu
 - Plusieurs modes de jeu (joueur contre joueur ou ia, jeu local, en réseau)
 - 2 interfaces utilisateur (CLI & GUI)
 - Application modulaire, maintenable et testable
 - Base pour l'extension des règles, amélioration IA et l'évolution du mode réseau
- Contraintes :
 - Langage : Java; Système cible : GNU/Linux
 - Représentation du plateau : bitboards
 - Interfaces obligatoires (CLI interactive & GUI JavaFX)
 - Fonctionnalités obligatoires (undo/redo, etc...)
 - IA & Réseau
 - Qualité logicielle (tests ...)

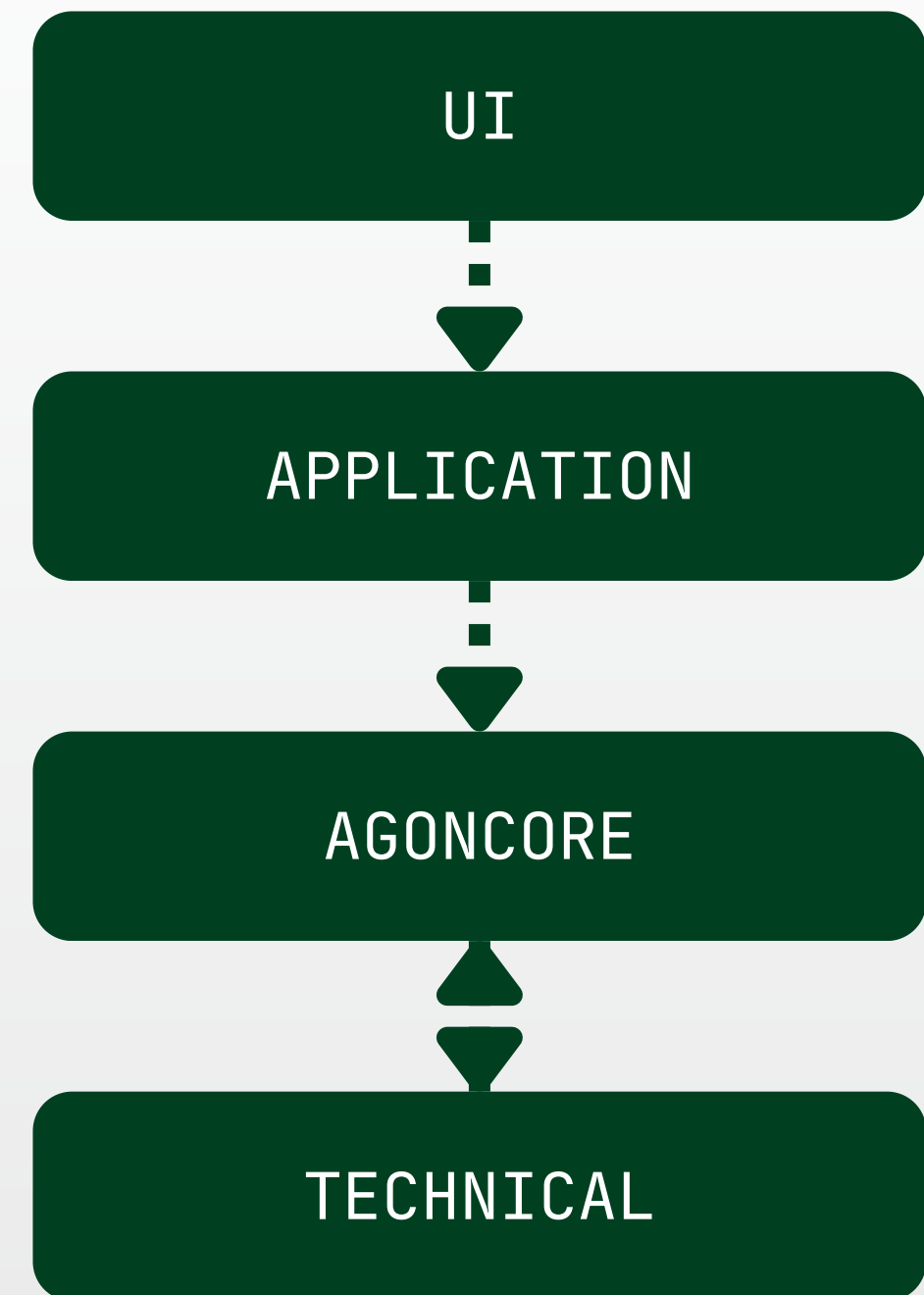


AGON Répartition du travail



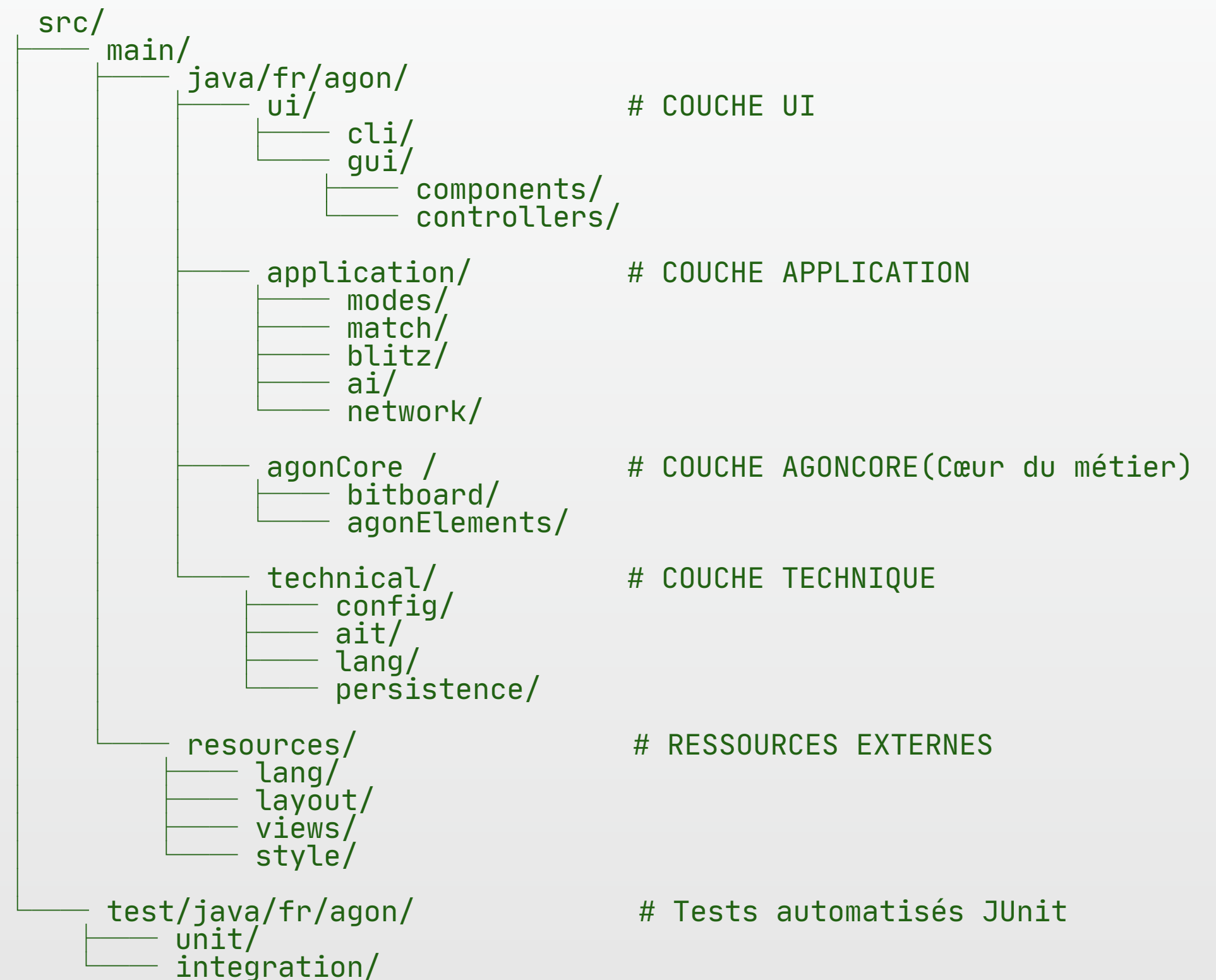
AGON Architecture du logicielle

- Choix architecture : Architecture en couches
- Couches:
 - UI : gérer l'interaction avec l'utilisateur
 - Application : diriger les actions du système
 - AgonCore : définir ce qu'est le jeu Agon
 - Technical : gérer les aspects techniques
- Relations entre les couches:
 - UI -> Application : Transmet les intentions de l'utilisateur.
 - Application -> AgonCore : Exécute la logique métier et les règles du jeu.
 - AgonCore <-> Technical : Échange des données via des interfaces.



AGON Arborescence globale du projet

- Représentation de nos 4 couches :
 - "ui"
 - "application"
 - "agonCore"
 - "technical"
- Dossier "resources" & "test"

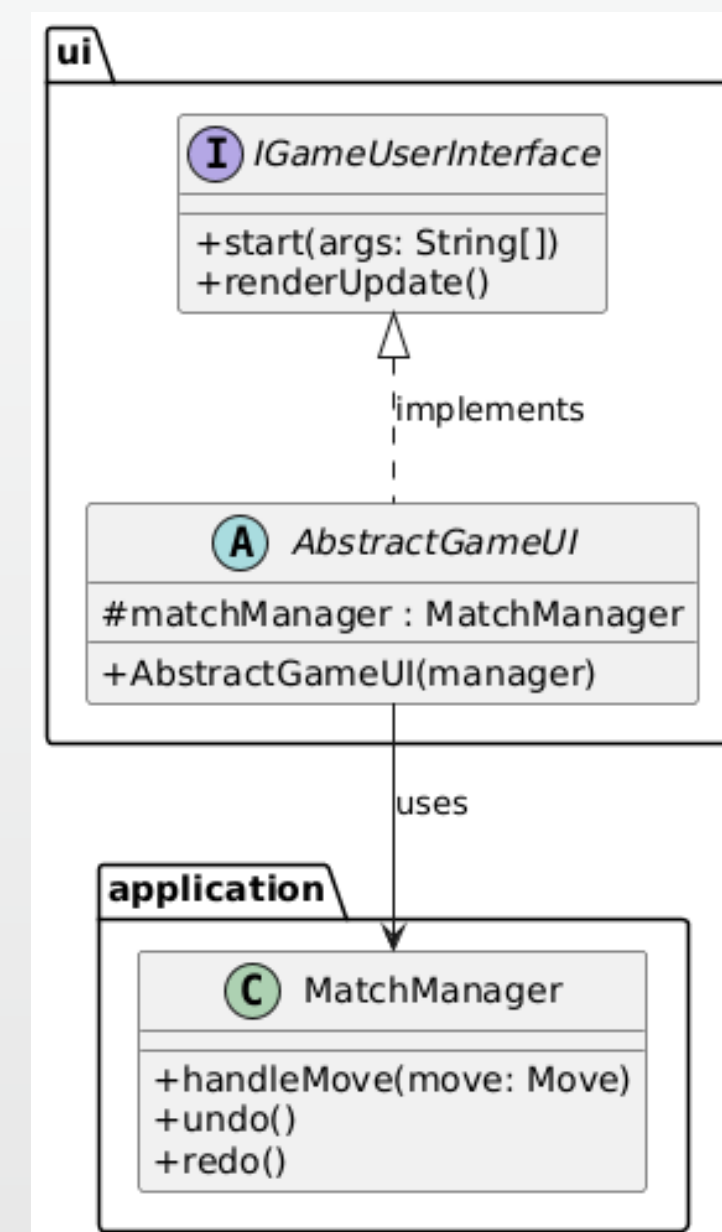


AGON

Zoom sur couches: UI

- Rôle fichier dans dossier "ui" :
Définit le contrat d'interaction (IGameUserInterface)
pour garantir l'indépendance entre le moteur de jeu et
l'affichage.
- Dossier :
 - "cli" : shell interactif, lecture des commandes,
affichage texte
 - "gui" : interface graphique du jeu
 - "gui/controllers" : gestionnaire d'événements
 - "gui/components" : composants JavaFX personnalisés

```
ui/      # COUCHE PRÉSENTATION
|       IGameUserInterface.java # Interface
|       AbstractGameUI.java   # Classe Abstraite
|       cli/
|       gui/
```



AGON

Zoom sur couches: UI - CLI

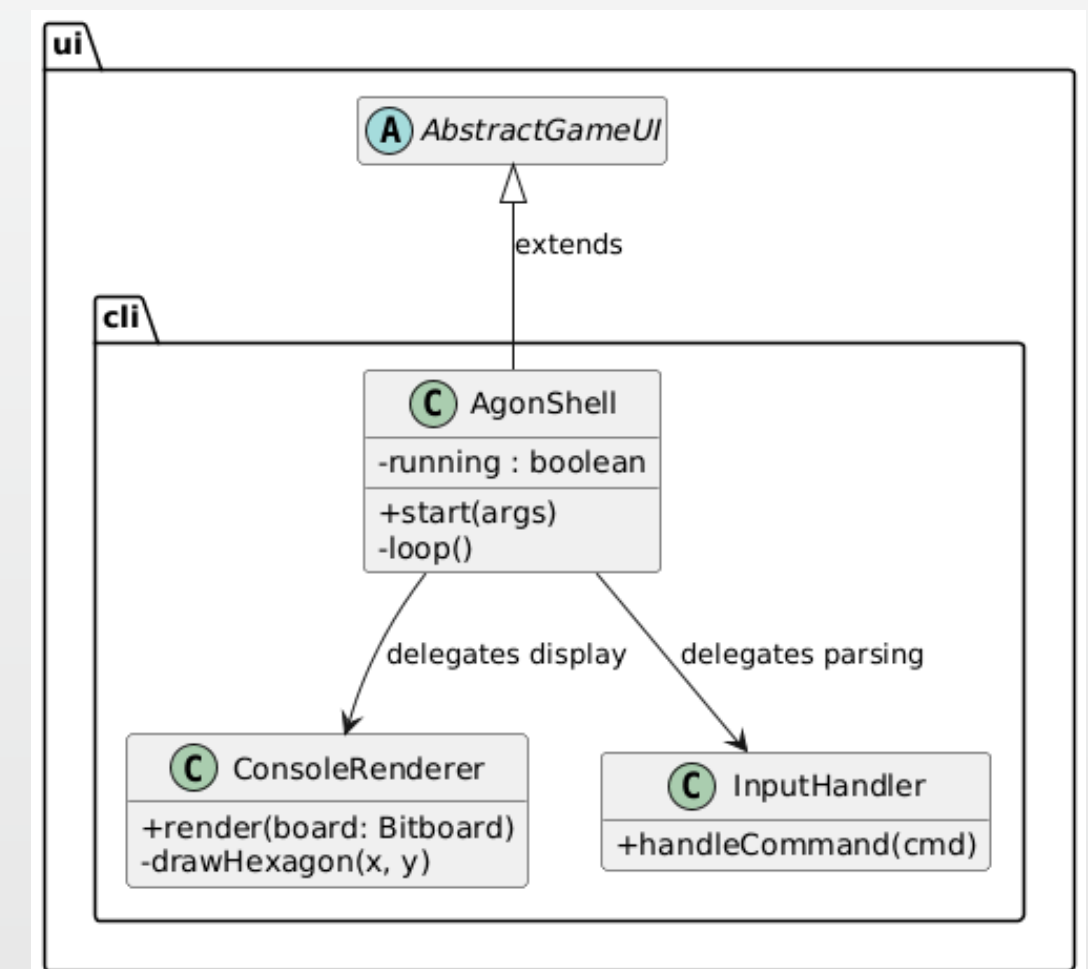
Réponse aux Besoins (Cahier des Charges) :

- Shell Interactif (F15) : Implémentation d'une boucle de lecture (REPL) via Scanner/JLine.
- Notation (F20) : Parsing des commandes textuelles (ex: move c4 d5).
- Rendu ASCII (F24) : Affichage du plateau compatible avec tout terminal standard.

Solution Technique :

- AgonShell : Implémente l'interface commune.
- ConsoleRenderer : Séparation de la logique d'affichage. Convertit l'état des Bitboards en grille de caractères (x, o, .).

```
cli/  
├── AgonShell.java # Boucle principale (extends  
AbstractGameUI)  
├── ConsoleRenderer.java # Moteur de rendu ASCII  
└── InputHandler.java # Gestion du parsing des  
commandes
```



AGON

Zoom sur couches: UI – GUI

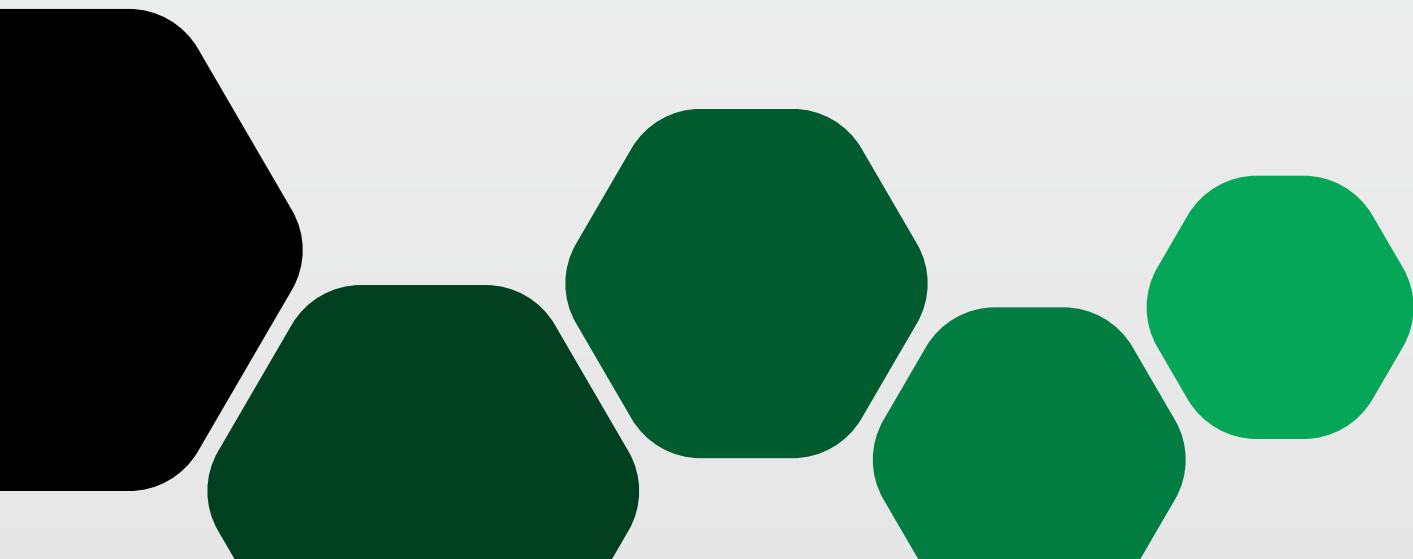
Réponse aux Besoins (Cahier des Charges) :

- Expérience Utilisateur (F30) : Déplacement des pièces par Drag & Drop (Souris).
- Contrôle (F28/F29) : Barre de menus et raccourcis clavier configurables.

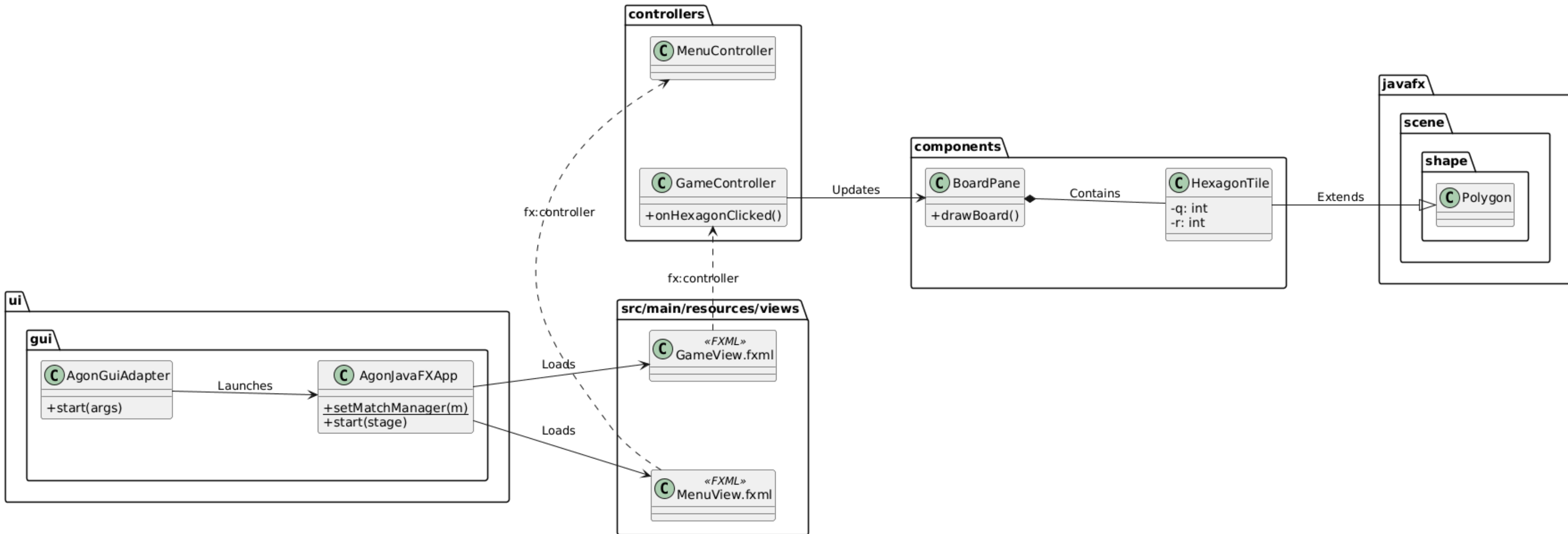
Solution Technique :

- AgonGuiAdapter : Fait le pont entre notre interface commune et le cycle de vie complexe de JavaFX.
- Composants Custom (HexagonTile) : Héritage de Polygon pour une détection de clic précise.

```
gui/  
├── AgonGuiAdapter.java  
├── AgonJavaFXApp.java # Classe principale JavaFX.  
├── controllers/  
│   ├── GameController.java #Gestion du jeu  
│   └── MenuController.java #Gestion du menu  
└── components/  
    ├── HexagonTile.java # Composant Polygon  
    └── BoardPane.java  
  
resources/  
└── views/  
    ├── GameView.fxml # Description du jeu  
    └── MenuView.fxml # Description du menu
```



AGON Zoom sur couches: UI - GUI

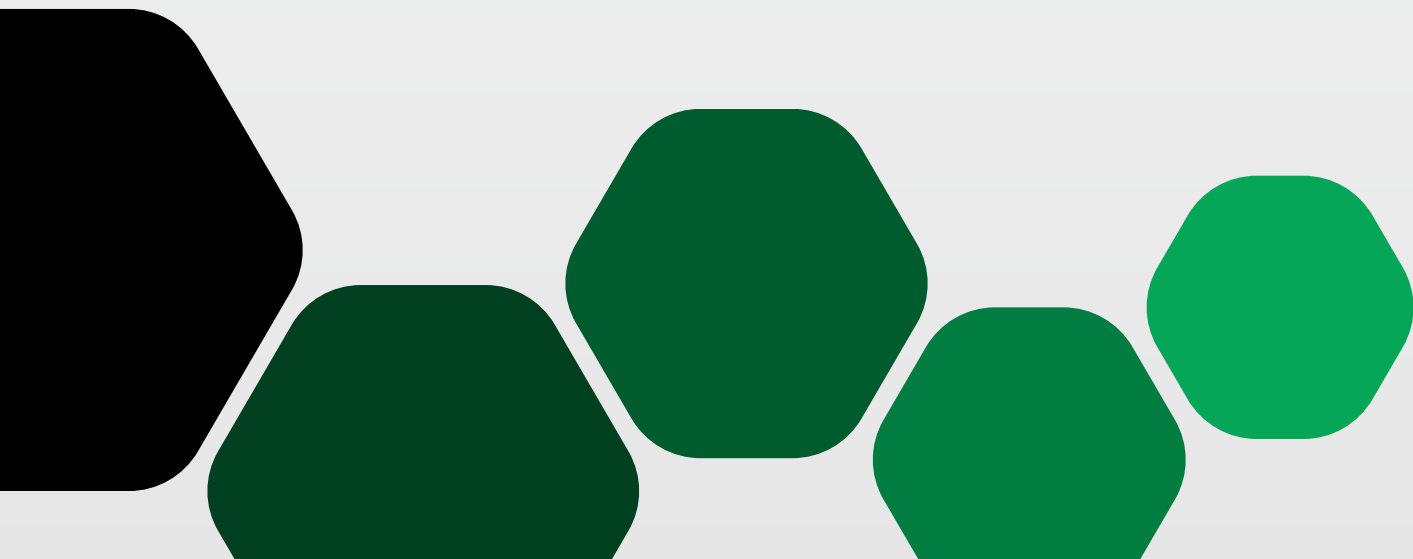


AGON

Zoom sur couches: Application

- Rôle fichier dans dossier "application" :
"cerveau" opérationnel du projet
- "MatchManager" => dirige le jeu
- Dossier :
 - "modes" : définit comment le jeu est lancé et exécuté selon le mode choisi.
 - "match" : pilote le déroulement d'une partie
 - "blitz" : gère les contraintes de temps et les comptes à rebours spécifiques pour le mode blitz
 - "ai" : implémente l'intelligence artificielle
 - "network" : supervise les communications distantes

```
application/      # COUCHE APPLICATION
├── MatchManager.java
├── modes/
├── match/
├── blitz/
├── ai/
└── network/
```

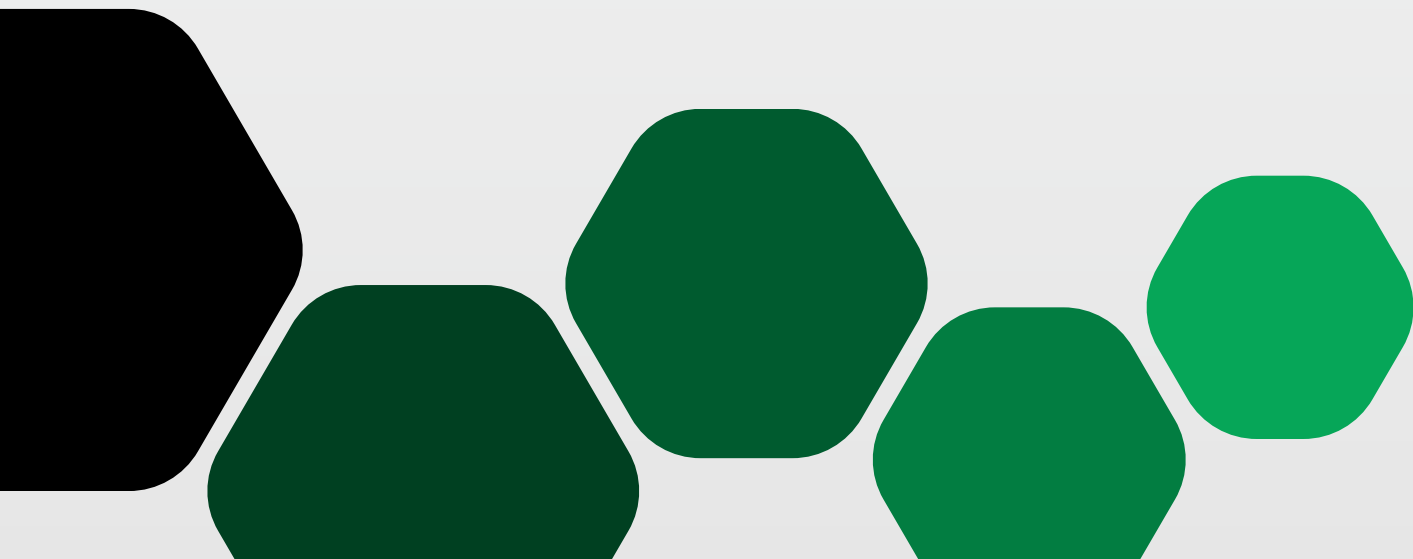


AGON

Zoom sur couches: Application – modes/match/blitz

- “modes/” : Scénarios de lancement
 - Rôles:
 - Centralise la logique de démarrage du jeu
 - Sépare les différents comportements d'exécution
- “match/” : Pilote le déroulement d'une partie
 - Rôles:
 - l'enchaînement des tours
 - l'historique, undo / redo
- “blitz/” : Contraintes temporelles
 - Rôles:
 - Chronomètre par joueur, Pause / reprise du jeu
 - Arrêt automatique de la partie au dépassement du temps

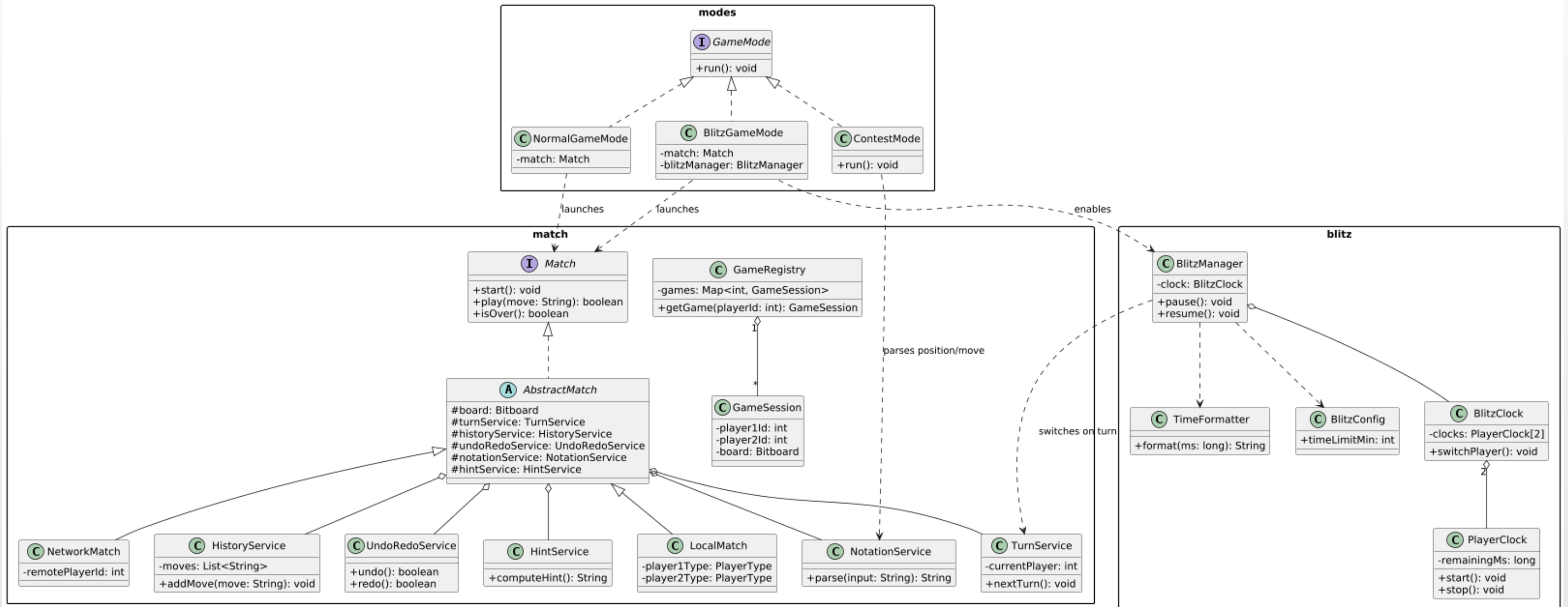
```
application/  
├── MatchManager.java  
├── modes/  
│   ├── GameMode.java  
│   ├── NormalGameMode.java  
│   ├── BlitzGameMode.java  
│   └── ContestMode.java  
├── match/  
│   ├── Match.java  
│   ├── AbstractMatch.java  
│   ├── LocalMatch.java  
│   ├── NetworkMatch.java  
│   ├── GameSession.java  
│   ├── GameRegistry.java  
│   ├── TurnService.java  
│   ├── HistoryService.java  
│   ├── UndoRedoService.java  
│   ├── NotationService.java  
│   └── HintService.java  
└── blitz/  
    ├── BlitzConfig.java  
    ├── PlayerClock.java  
    ├── BlitzClock.java  
    ├── BlitzManager.java  
    └── TimeFormatter.java
```



AGON

Zoom sur couches:

Application - modes/match/blitz

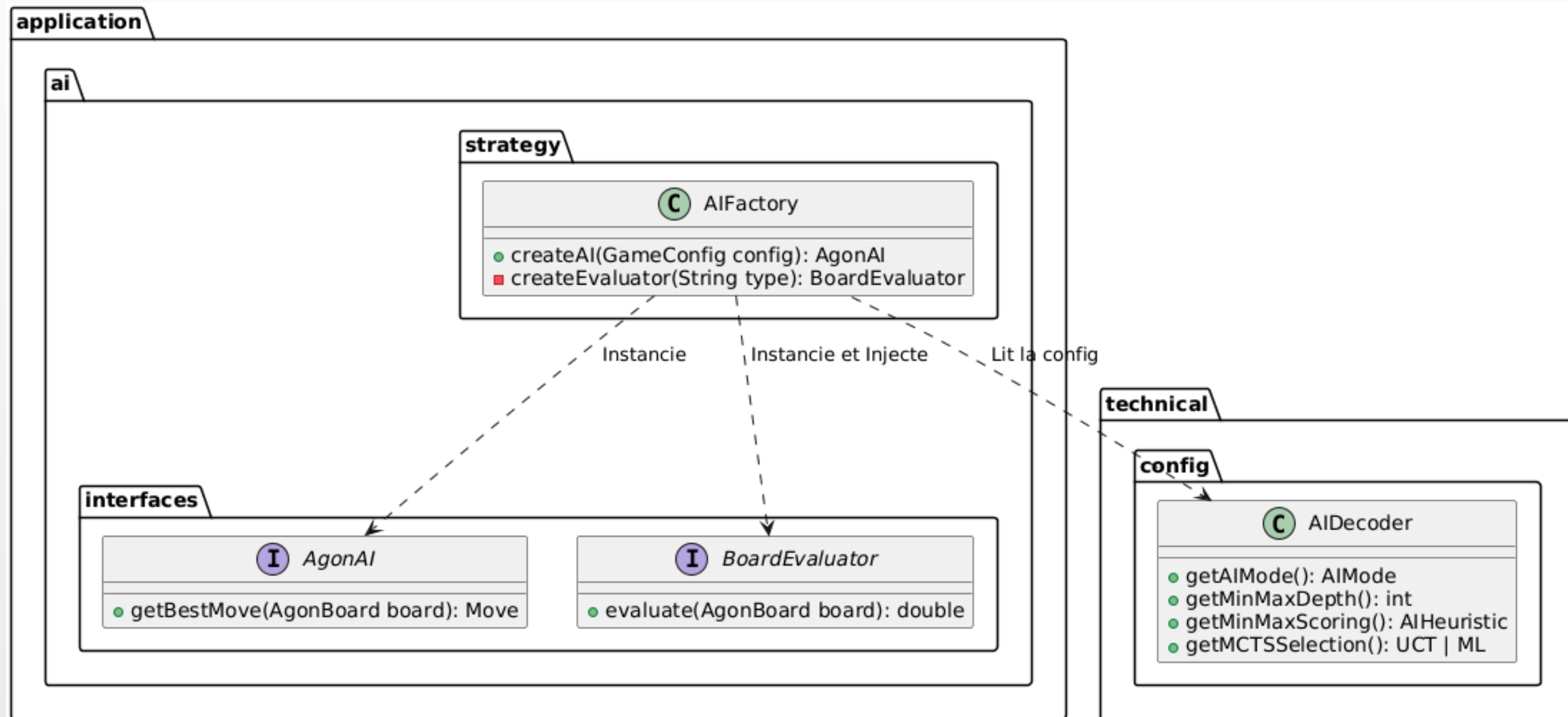


AGON Zoom sur couches: Application - IA

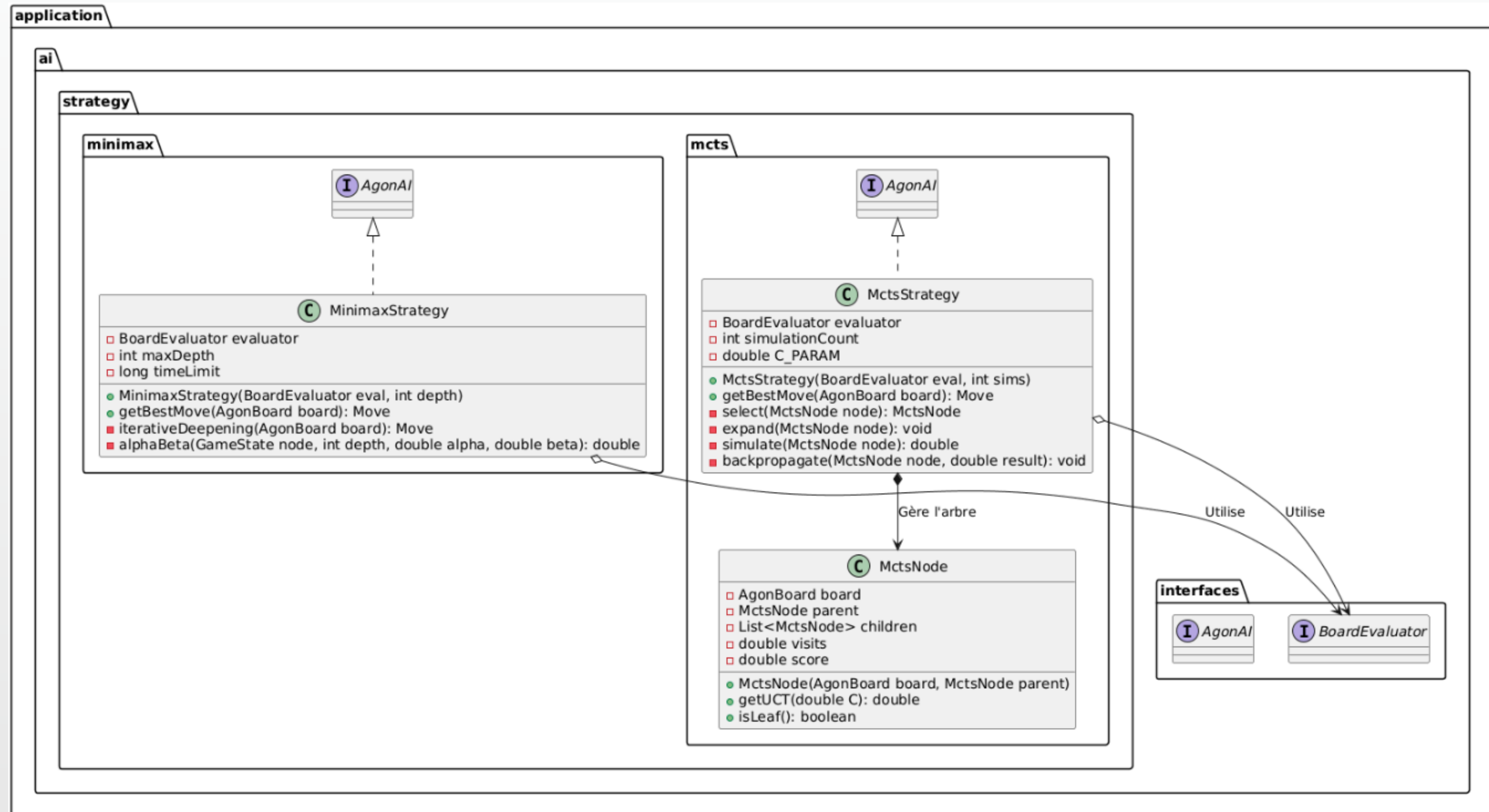
- Organisation : Architecture modulaire basée sur le pattern Strategy et l'Injection de Dépendances.
- Interfaces :
 - AgonAI : Contrat unique pour jouer un coup.
 - BoardEvaluator : Factorisation majeure. Interface commune utilisée par Minimax (pour les feuilles) et MCTS (pour la simulation/sélection).
- Stratégies :
 - Séparation nette entre l'algorithme de recherche (strategies) et la fonction d'évaluation (heuristics).
 - AIFactory : Assemble dynamiquement l'IA selon la config.
- Isolation Technique :
 - Le Machine Learning (MLHeuristic) est une heuristique comme les autres.
 - Dépendance vers technical/ait pour l'appel TensorFlow.

```
src/main/java/fr/agon/  
├── application/  
│   ├── ai/  
│   │   ├── interfaces/  
│   │   │   ├── AgonAI.java  
│   │   │   └── BoardEvaluator.java  
│   │   ├── strategy/  
│   │   │   ├── AIFactory.java  
│   │   │   ├── minimax/  
│   │   │   │   └── MinimaxStrategy.java  
│   │   │   └── mcts/  
│   │   │       ├── MctsStrategy.java  
│   │   │       └── MctsNode.java  
│   │   └── heuristics/  
│   │       ├── CentralityHeuristic.java  
│   │       ├── MobilityHeuristic.java  
│   │       ├── MixedHeuristic.java  
│   │       └── MLHeuristic.java  
└── technical/  
    └── ait/  
        └── TensorFlowAdapter.java
```

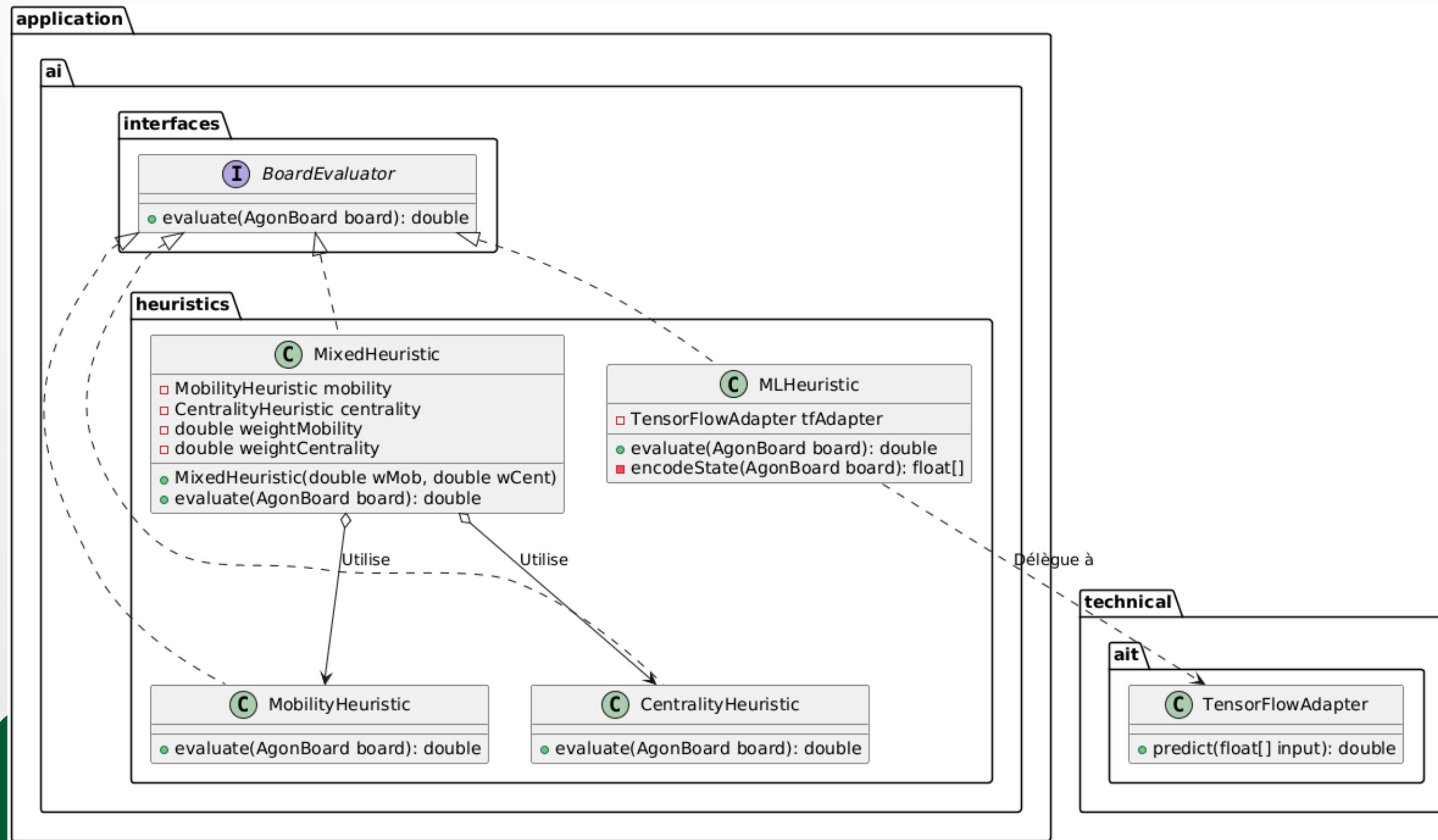
AGON IA : Construction dynamique



AGON IA : Factorisation des stratégies



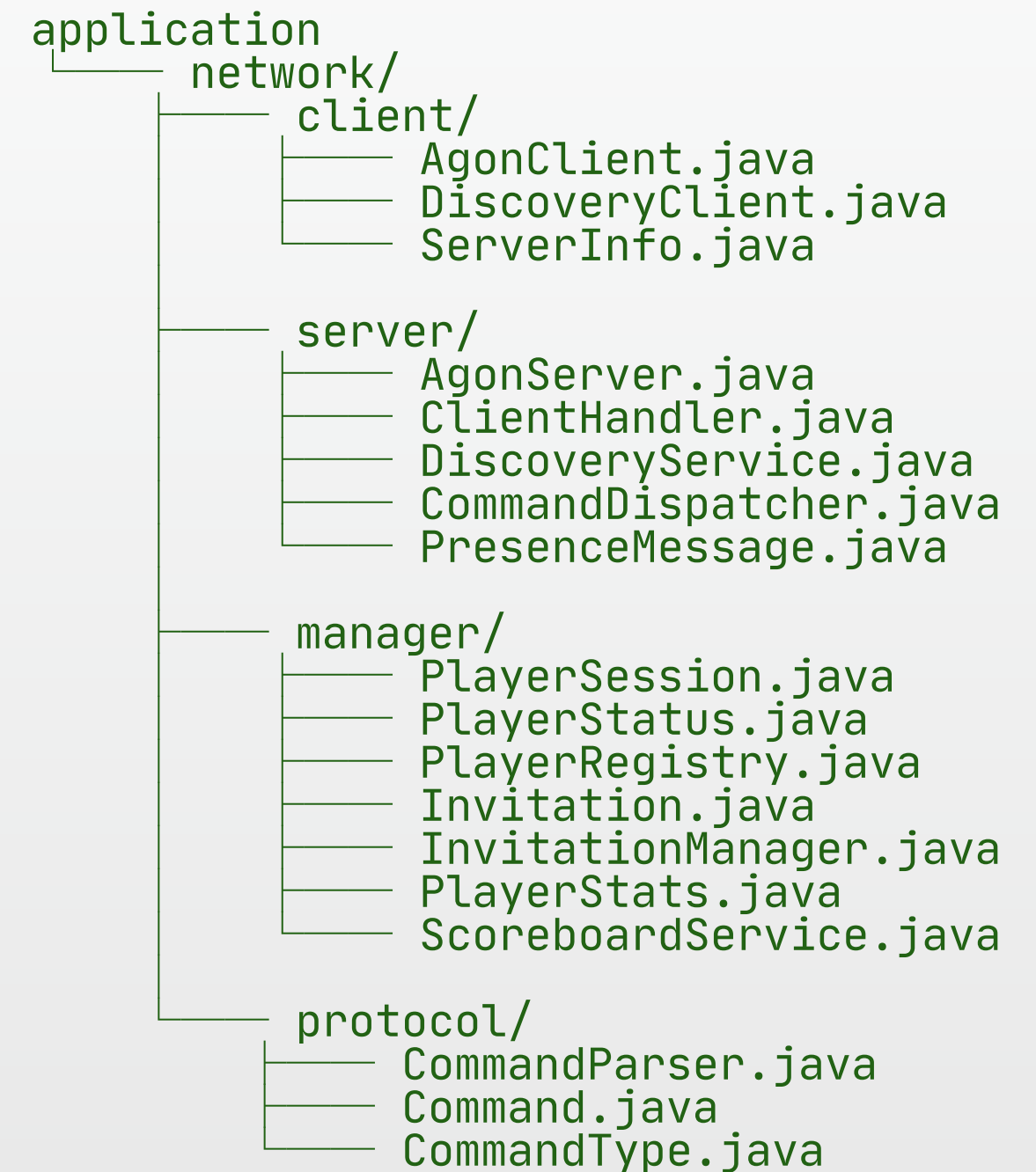
AGON IA : Couche d'évaluation et isolation du ML



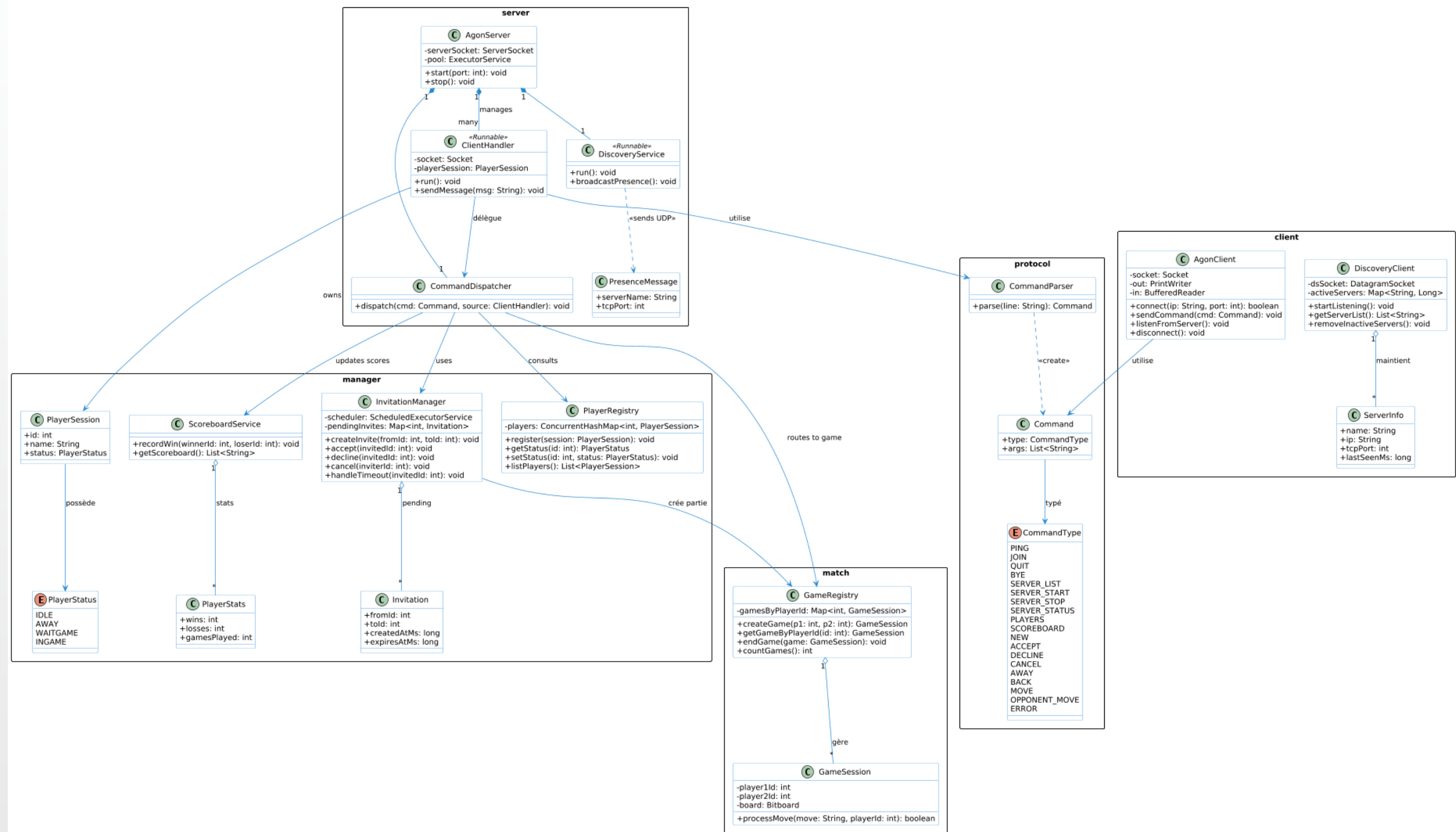
AGON

Zoom sur couches: Application - Network

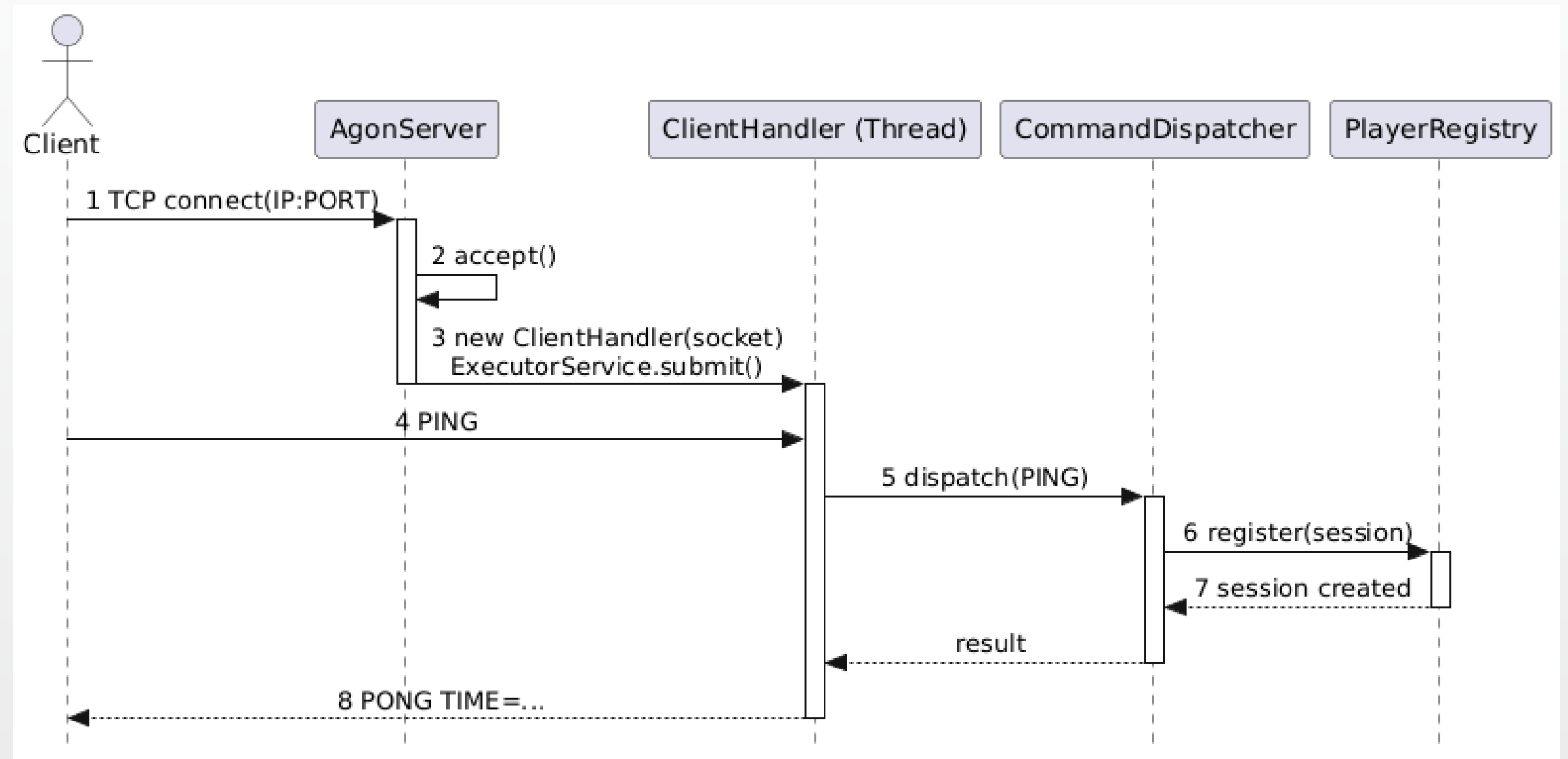
- Organisation en couches :
 - client & server : Couche d'accès (Technique pure Sockets)
 - manager : Couche métier (Gestion des états des joueurs et des invitations)
 - protocol : Couche de traduction (Parsing via méthodes String)
- Gestion des Flux & Parallélisme :
 - Multi-threading : Utilisation d'un ExecutorService pour gérer plusieurs clients en simultané sans bloquer le serveur.
 - Synchronisation : Utilisation de Mutex / synchronized pour protéger les données partagées (liste des scores, statut des joueurs) contre les accès concurrents.
- Gestion du Temps :
 - Timeouts : Emploi de ScheduledExecutorService pour automatiser l'expiration des invitations



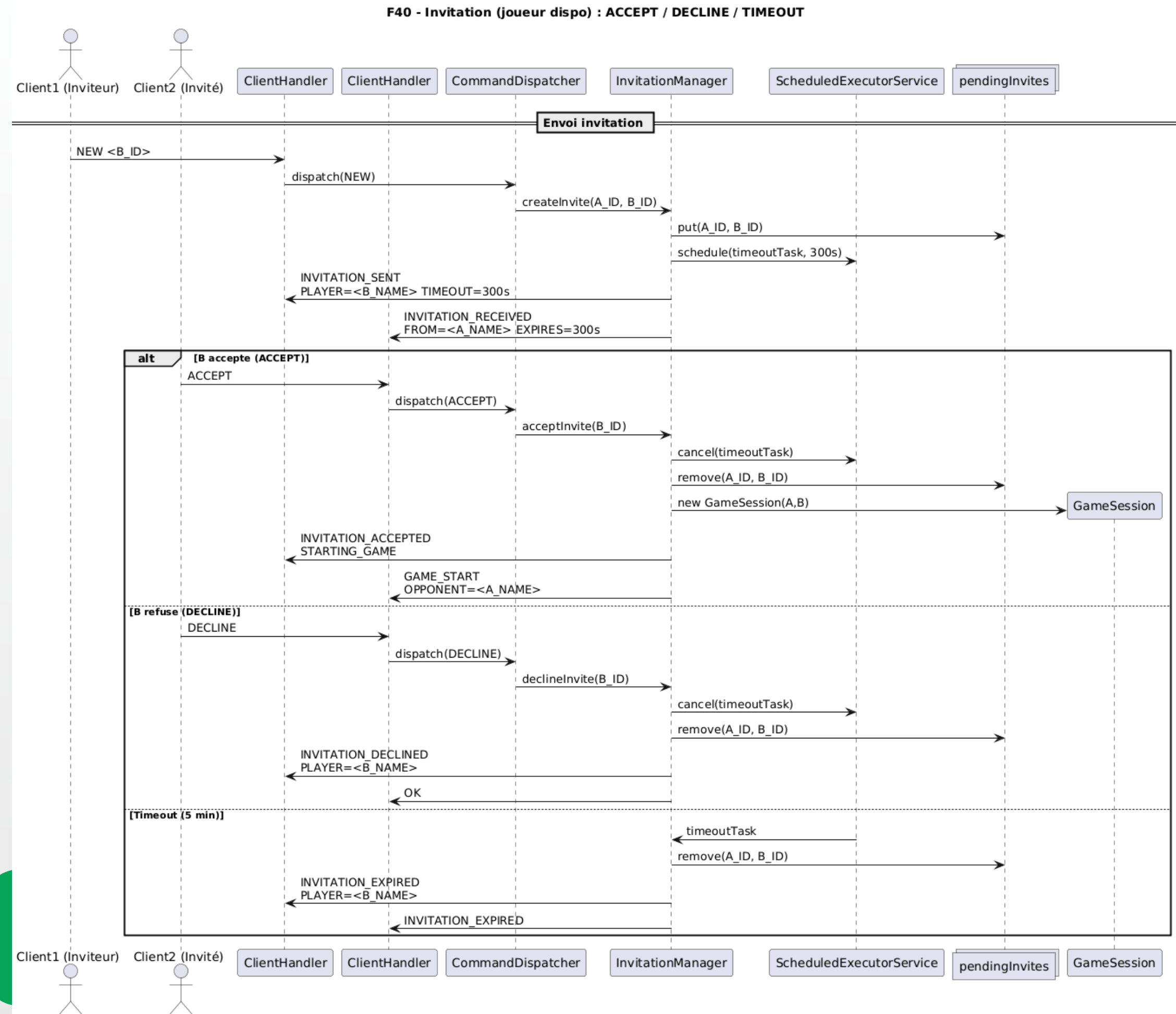
AGON



AGON



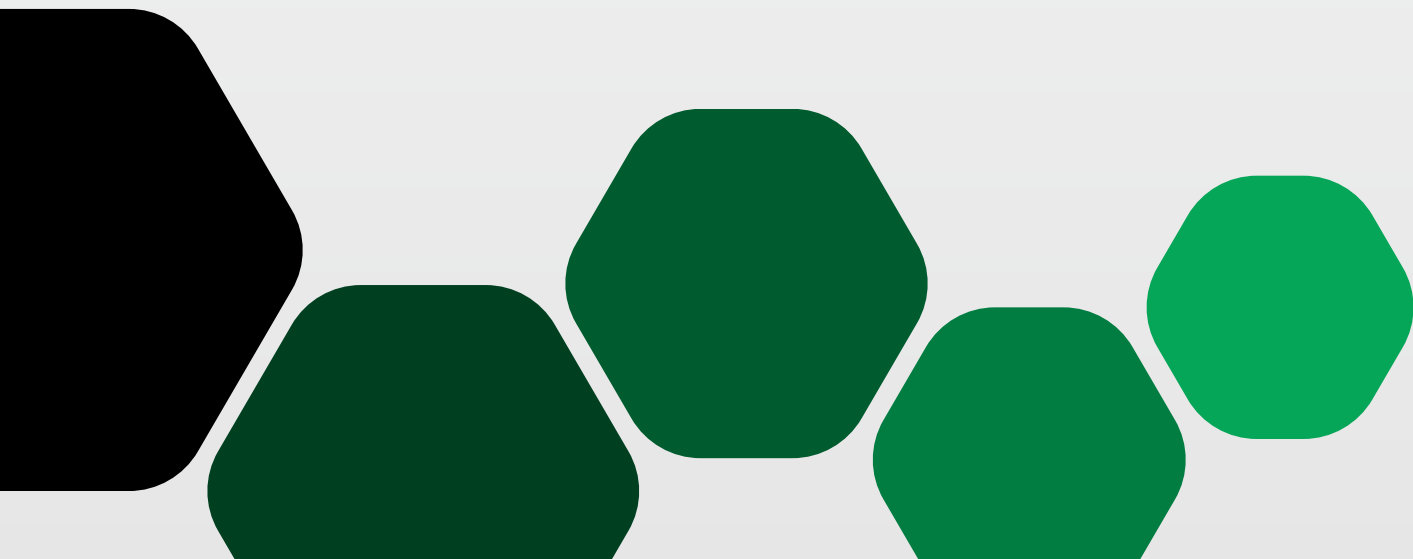
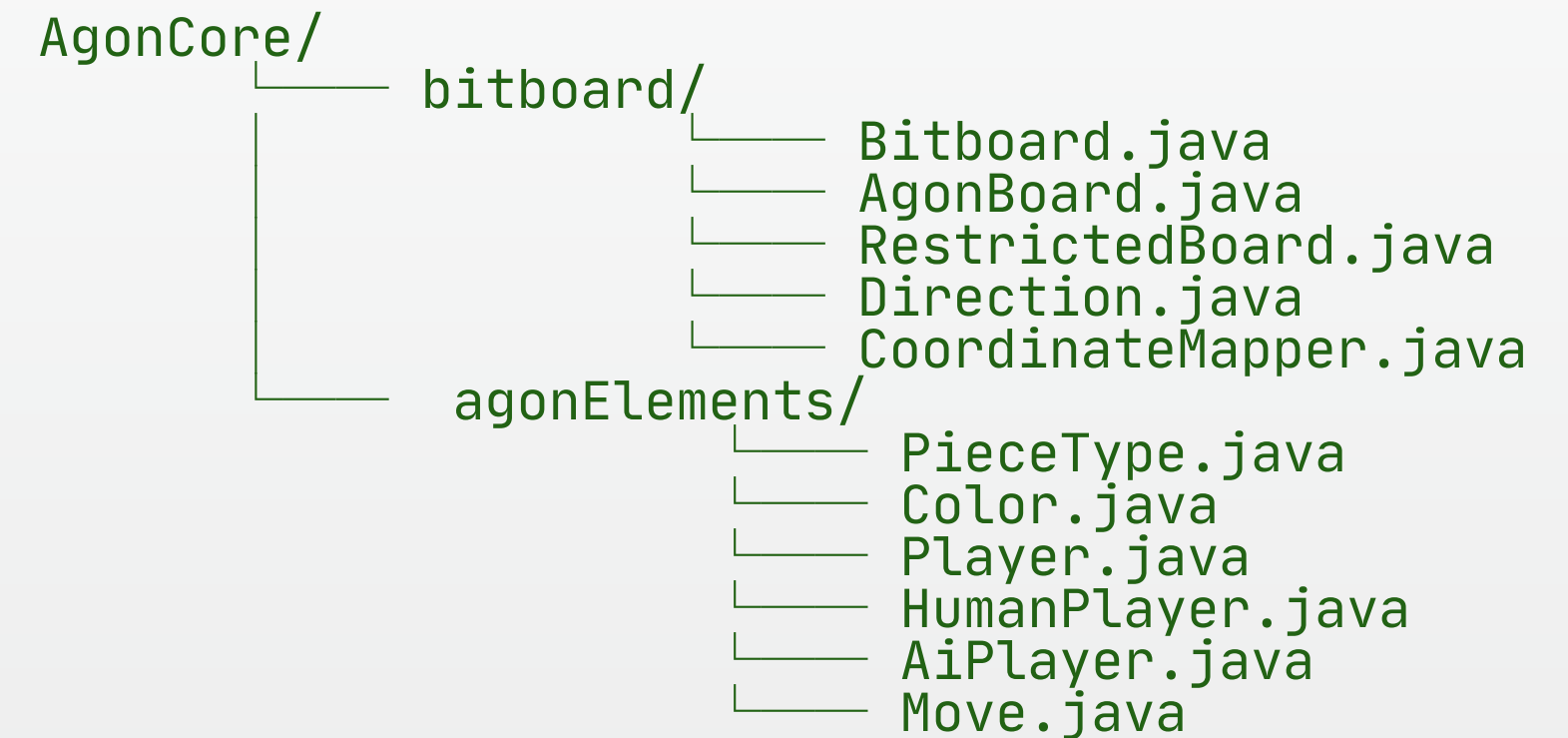
AGON



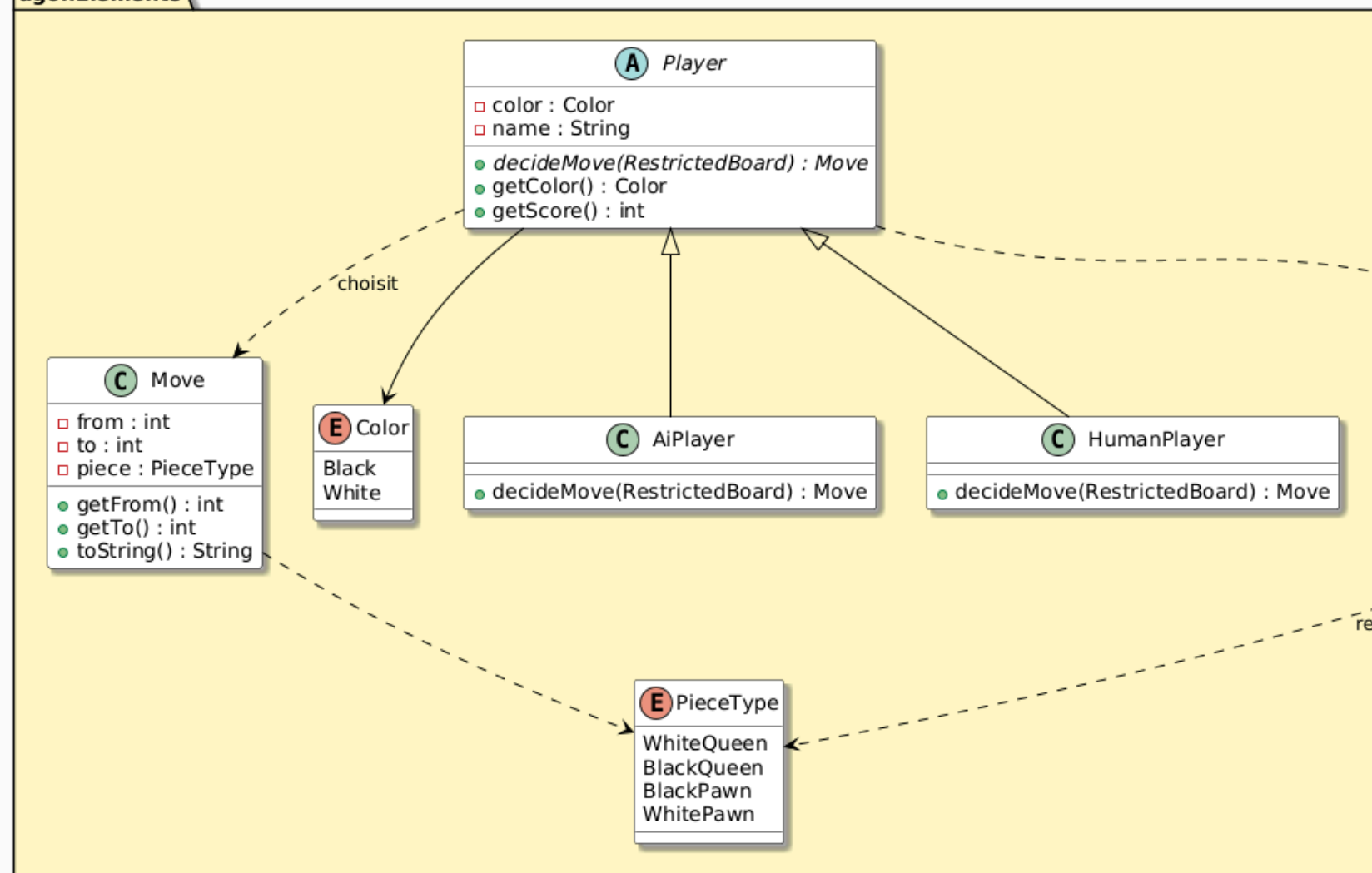
AGON

Zoom sur couches: Partie AgonCore

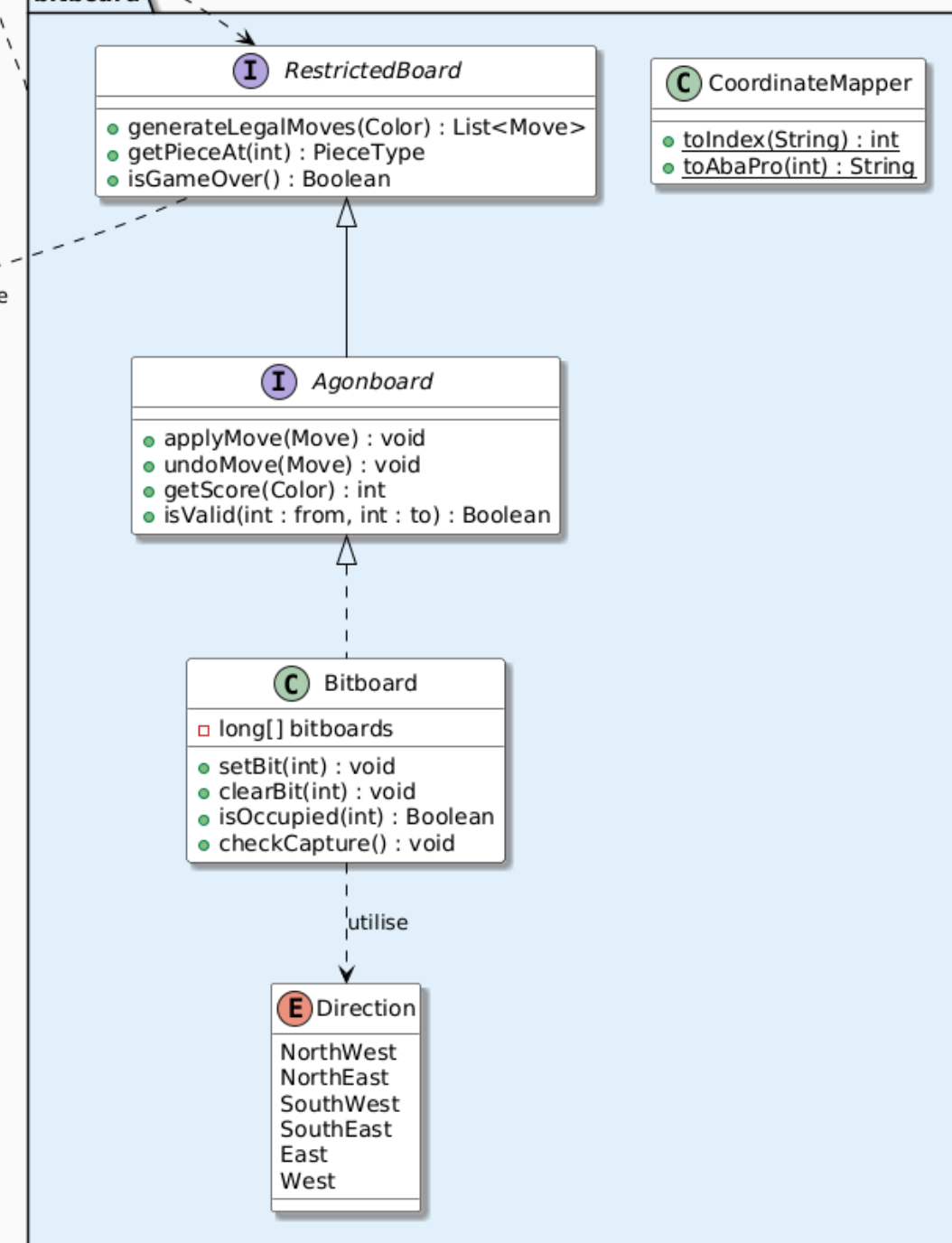
- Dossier :
 - bitboard : Structure de données bitboard, encode l'état du plateau et algorithmes de manipulation.
 - AgonElements : ensemble de représentation métier nécessaire à l'application
- Résumé technique :
 - Opérations bitwise



agonElements



bitboard



AGON Zoom sur couches: Technical

- Rôle fichier dans dossier "technical" :
Gère les outils techniques
- Dossier :
 - "config" : gère la lecture et l'analyse du fichier .agonrc
 - "lang" : fournit le support multilingue (i18n)
 - "storage" : assure la persistance des données
 - "ai" : contient toute la "pollution" technique liée à la bibliothèque externe (TensorFlow, etc...)

technical/ # COUCHE INFRASTRUCTURE

- Serializer (transforme les objets en texte)
- Parser (transforme le texte en objets)
- config/
- storage/
- ai/
- lang/

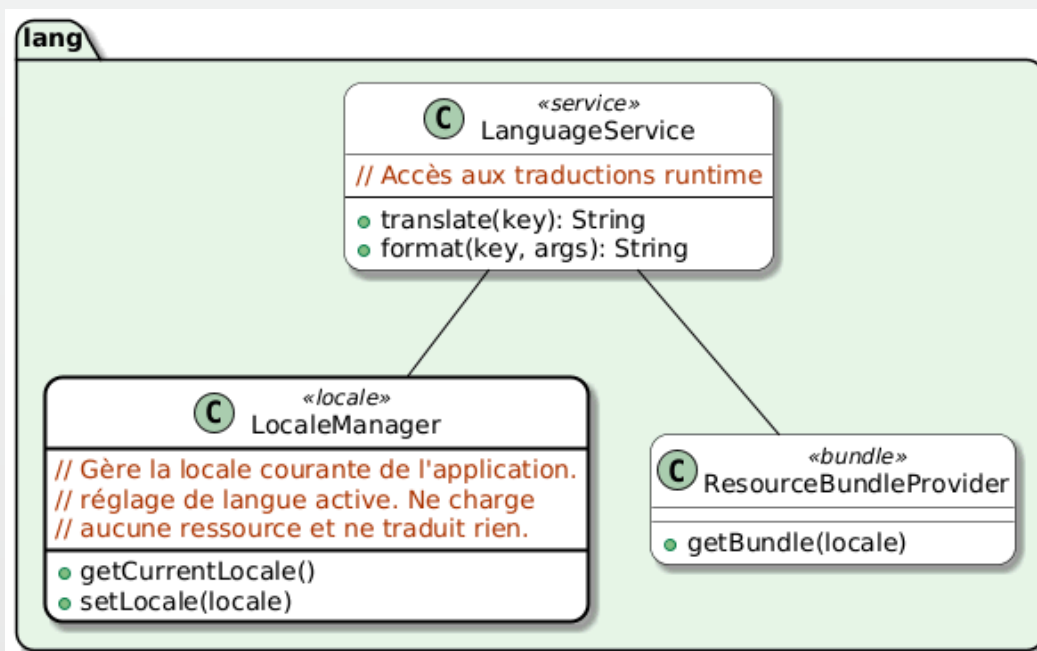
«serializer»
 TextSerializer
● serialize(obj)

«parser»
 TextParser
● parse(text)

AGON

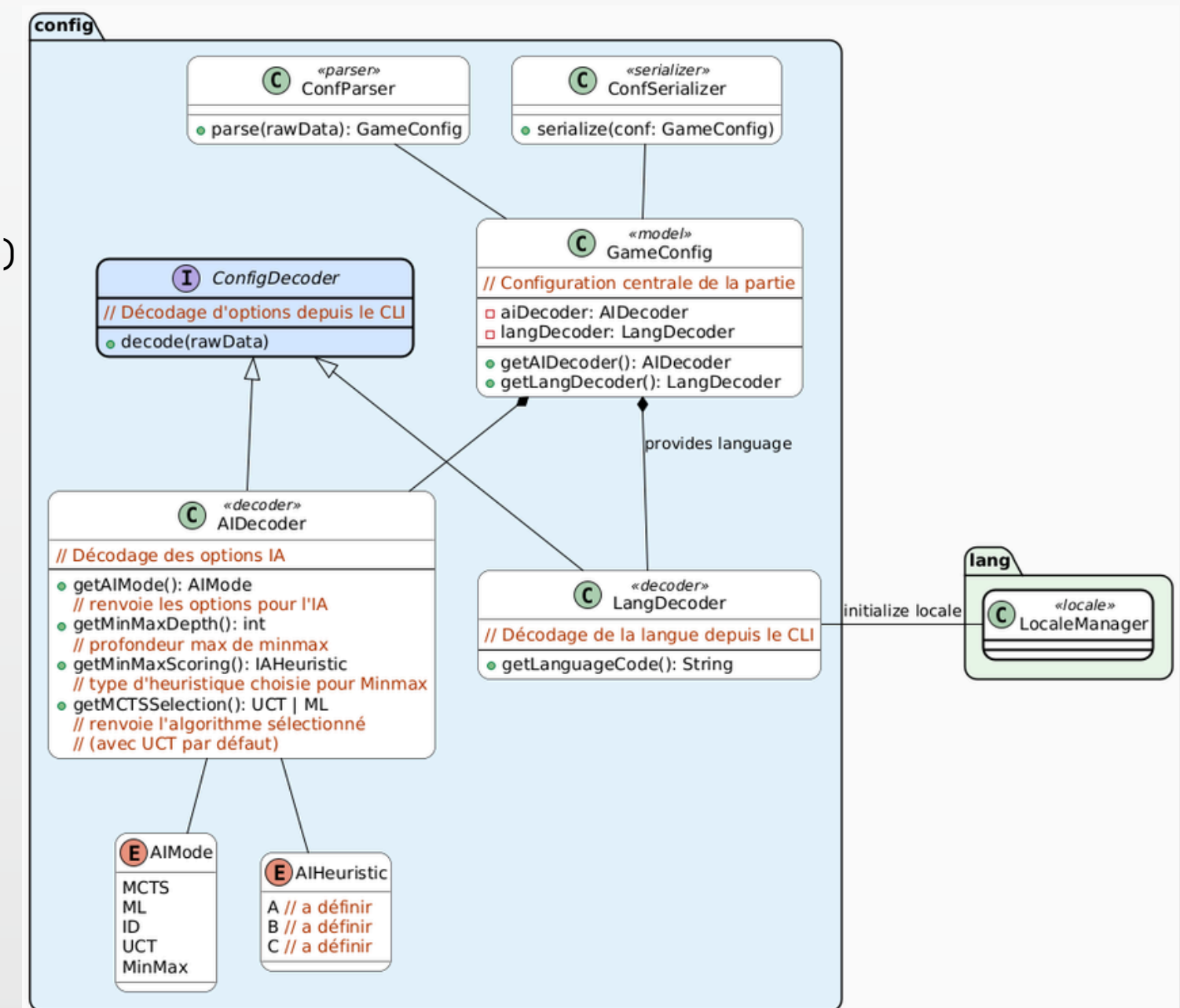
Zoom sur couches: Technical - Lang & Configuration

```
technical/  
├── config/  
│   ├── ConfSerializer ""  
│   ├── ConfParser ""  
│   ├── GameConfig (Représenter la configuration du jeu, issue du fichier .agonrc)  
│   ├── LangDecoder (gestion des options de langue)  
│   ├── ConfigDecoder (gère la récupération et le traitement des options du CLI)  
│   ├── AIDecoder (se charge des options des IAs)  
│   └── AIMode et AIHeuristique explicite je pense  
└── lang/  
    ├── bundle (fournit l'accès aux ressources multilingues brutes manip des données .json)  
    ├── locale (gère la langue courante des param régionaux)  
    └── service (Interface centralisée pour l'accès à la traduction)
```



Couleur	Rôle
	Configuration (.agonrc)
	Internationalisation (i18n)
	Persistance technique et IA
	Technical (parsing / serialization générique)
	IA (framework externe, TensorFlow)

- Interfaces et classes spécifiques sont séparées
- Aucune dépendance métier dans technical, persistance ou IA
- Les packages techniques peuvent être mockés pour les tests



AGON

Zoom sur couches: Technical

- Support & Persistence

technical/ # COUCHE INFRASTRUCTURE

 persistence/

 PRepository (role d'accès au données)

 PStorage (sauvegarde des données de l'IA)

 PDataSource (Accès physique aux données, gestion bas niveau des fichiers ou flux réseau)

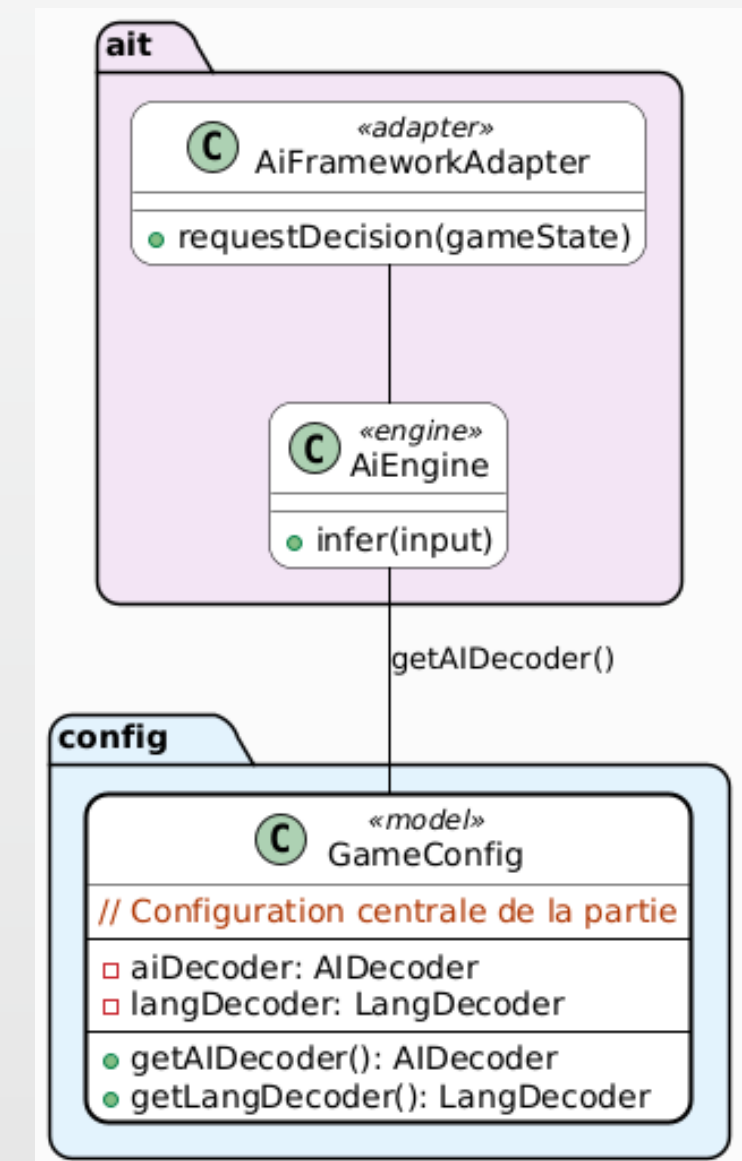
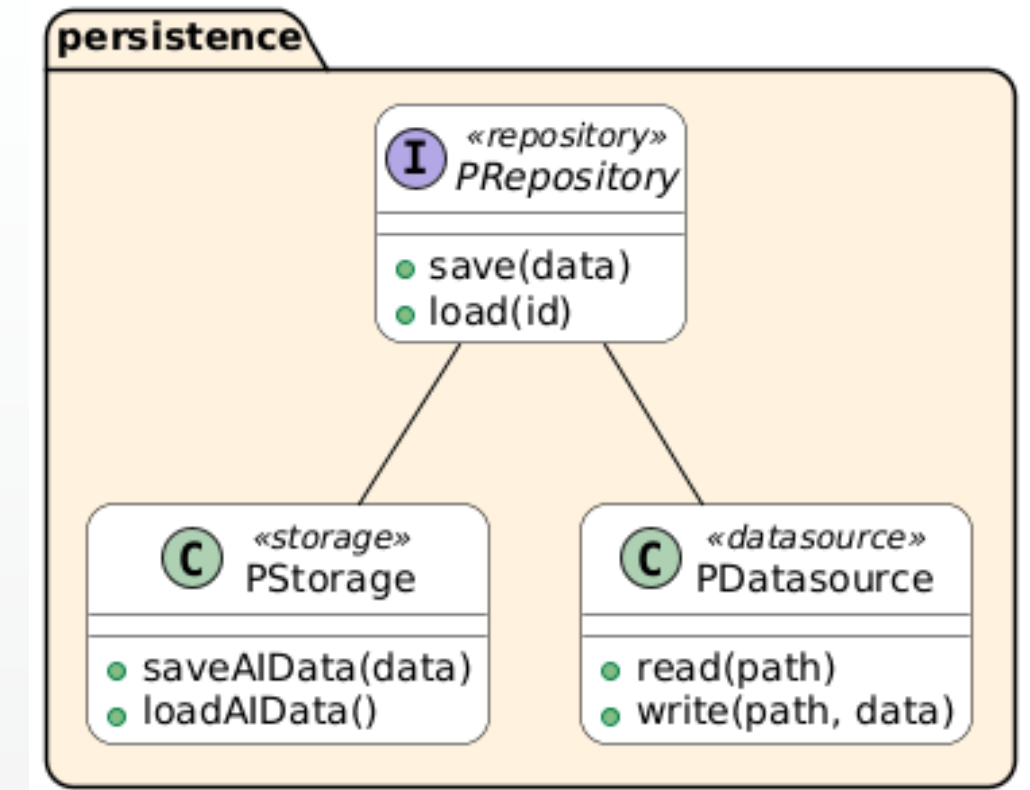
 ait/

 adapter (Design Pattern Adapter pour adapter la sortie TensorFlow aux données)

 engine (effectue la logique des calcul/opération avec TensorFlow)

Couleur	Rôle
	Configuration (.agonrc)
	Internationalisation (i18n)
	Persistence technique et IA
	Technical (parsing / serialization générique)
	IA (framework externe, TensorFlow)

- Interfaces et classes spécifiques sont séparées
- Aucune dépendance métier dans technical, persistence ou IA
- Les packages techniques peuvent être mockés pour les tests



AGON Tests & performances

- Tests:

- Objectif : vérifier la conformité aux règles du jeu et la robustesse du logiciel
- Outils prévus : JUnit (tests automatisés); JaCoCo (mesure de la couverture)
- Critères de validation : $\geq 85\%$
- Tests unitaires (cœur métier)
- Tests d'intégration
- Tests réseau
- Tests de non-régression

- Performances:

- Objectif : garantir des temps de réponse acceptables pour le joueur
- Mesures prévues : temps d'exécution de l'IA, latence réseau ...
- Cas limites: historique long, plusieurs clients => même serveur

tests

└─ duplication de package de la zone src/
└─ performances

└─ Measures (mesure des performances (IA, réseau, cas limites))

└─ IA (tests de temps d'exécution et optimisation des algos IA)

└─ Network (tests de latence et stabilité multi-clients)

└─ EdgeCases (tests sur scénarios limites : historique long, plusieurs joueurs simultanés)

